

[Name der Hochschule]

[Art der Arbeit]

[Titel der Fallstudie zum Technologie-Template]

[Untertitel]

[Aufgabenstellung]

Kurs: [Kurs / Modul]
Studiengang: [Studiengang]
Autor: [Vorname Nachname]
Matrikelnummer: [Matrikelnummer]
Tutor: [Tutor / Betreuer]
Abgabedatum: [Monat Jahr]

Inhaltsverzeichnis

Abbildungsverzeichnis	IV
Tabellenverzeichnis	V
Abkürzungsverzeichnis	VI
1 Einleitung	1
1.1 Ausgangslage	1
1.2 Problemstellung	1
1.3 Zielsetzung	1
1.4 Fragestellungen	2
1.5 Methodisches Vorgehen	2
1.6 Aufbau der Fallstudie	3
2 Anforderungen an das Technologie-Template	4
2.1 Zielbild des Technologie-Templates	4
2.2 Anforderungen an Wiederverwendbarkeit und Standardisierung	4
2.3 Anforderungen für Web-, Cloud- und Desktop-Anwendungen	5
2.4 Qualitäts- und Wartbarkeitsanforderungen	5
2.5 Abgrenzung	5
3 Architektur des Technologie-Templates	6
3.1 Gesamtarchitektur	6
3.2 Frontend mit Vite und TypeScript	8
3.3 Backend mit FastAPI	8
3.4 Desktop-Integration mit Tauri und Rust	9
3.5 Shared Contracts und Schnittstellen	10
3.6 Konfigurationsmodell	10
3.7 Persistenz mit PostgreSQL, SQLAlchemy und Alembic	11
4 Projektprofile und Feature-Modell	12
4.1 Grundprinzip des Profilmodells	12
4.2 Profil Web-only	14
4.3 Profil Web-cloud	14

4.4	Profil Desktop-local	15
4.5	Profil Desktop-cloud	15
4.6	Profil Full-platform	15
4.7	Optionale Capabilities	15
4.8	Single-Repository-Strategie	16
5	Tooling und Entwicklungsworkflow	17
5.1	Rolle von control.py	17
5.2	Projektinitialisierung und Generierung	18
5.3	Systemprüfung und Installation	19
5.4	Entwicklungsbetrieb	19
5.5	Tests und Builds	20
5.6	Release- und Packaging-Workflow	21
5.7	Entwickler-Onboarding	22
6	Qualitätssicherung und Verifikation	22
6.1	Teststrategie	22
6.2	Continuous Integration	23
6.3	Abnahme und technische Verifikation	24
6.4	Reproduzierbarkeit	25
6.5	Security und Konfigurationsgrenzen	26
6.6	Extern verbleibende Prüfungen	26
7	Bewertung des Technologie-Templates	27
7.1	Produktivitätswirkung	27
7.2	Stärken	28
7.3	Schwächen und Grenzen	29
7.4	Wartungsaufwand	30
7.5	Vergleich mit alternativen Template-Strategien	30
7.6	Gesamtbewertung	32
8	Einsatzmöglichkeiten und Transfer auf Kundenprojekte	33
8.1	Geeignete Projekttypen	33
8.2	Ungeeignete und eingeschränkt geeignete Projekttypen	35
8.3	Analyse von Kundenanforderungen	35

8.4	Auswahl des Projektprofils	36
8.5	Generierung eines Kundenprojekts	37
8.6	Überführung in die Produktentwicklung	38
9	Schlussbetrachtung	38
9.1	Zusammenfassung der Ergebnisse	38
9.2	Beantwortung der Fragestellungen	39
9.3	Kritische Würdigung	40
9.4	Ausblick	40
9.5	Fazit	41
	Literaturverzeichnis	42

Abbildungsverzeichnis

1	Methodische Schritte der Fallstudie	3
2	Gesamtarchitektur des Technologie-Templates	6
3	Komponenten und Verantwortungsgrenzen im Master Repository	7
4	Profilabhängige Laufzeitarchitektur	7
5	Providerneutrale Deployment-Architektur	9
6	Konfigurationspriorität und Sicherheitsgrenzen	11
7	Ableitung eines Produktprojekts aus einer gemeinsamen Template-Basis	12
8	Entscheidungsfluss für Profil und anschließende Capability-Auswahl	13
9	Strukturelle Feature-Zuordnung der fünf Profil-Presets	14
10	Implementierte Abhängigkeiten des Feature-Katalogs	16
11	Befehlsgruppen des zentralen Projekt-Toolings	17
12	Workflow der profilgesteuerten Projektgenerierung	18
13	Grundstruktur des Entwicklungsworkflows	20
14	Plattformübergreifendes Modell für Desktop-Releases	21
15	Nachweiskette der technischen Verifikation	23
16	Profilverifikation nach Evidenzstand und Commit	25
17	Qualitative Eignungsübersicht nach Architekturfit und Anpassungsbedarf	34
18	Vertikaler Transferprozess von der Kundenanforderung zum Produktprojekt	37

Tabellenverzeichnis

1	Zentrale Anforderungen an das Technologie-Template	4
2	Anforderungsmatrix der unterstützten Projektarten	5
3	Deklarierte und aufgelöste Technologie-Stack im untersuchten Commit	8
4	Deklarierte Basisfeatures und resultierende Laufzeitverzeichnisse	14
5	Profilbezogene technische Verifikation	25
6	Offene und externe Verifikationspunkte auf a4487ac	27
7	Evidenzbasierte Gegenüberstellung von Stärken und Grenzen	29
8	Qualitativer Vergleich von Template-Strategien	31
9	Zusammengeführte Bewertung des Technologie-Templates	32
10	Zuordnung technischer Projekttypen zur Template-Baseline	34
11	Kompakte Checkliste zur technikneutralen Anforderungsanalyse	36

Abkürzungsverzeichnis

Abk. Bedeutung

1 Einleitung

1.1 Ausgangslage

Die Einrichtung neuer Softwareprojekte erfordert wiederkehrende Entscheidungen zu Architektur, Technologien, Schnittstellen, Konfiguration sowie Test-, Build- und Deployment-Prozessen. Werden diese Grundlagen für jedes Vorhaben unabhängig erstellt, entstehen unterschiedliche technische Ausgangslagen. Web-, Cloud- und Desktop-Anwendungen stellen zudem verschiedene Anforderungen an Komponenten und Bereitstellung. Bei plattformübergreifenden Projekten müssen gemeinsame Bestandteile deshalb klar von spezifischen Erweiterungen getrennt werden.

Eine standardisierte technische Basis kann wiederkehrende Entscheidungen bündeln und ein konsistentes Entwicklungsmodell bereitstellen. Gegenstand der Fallstudie ist das Projekt „Template-Projekte“, ein wiederverwendbares Full-Stack-Technologie-Template für Web-, Cloud- und Desktop-Projekte. Seine Ausgestaltung und Eignung werden im weiteren Verlauf systematisch untersucht (kleiveist, 2026k).

1.2 Problemstellung

Die wiederholte Initialisierung technisch ähnlicher Projekte erzeugt Aufwand außerhalb der produktspezifischen Entwicklung. Ohne gemeinsame Baseline können sich Projektstruktur, Werkzeugauswahl, Konfiguration und Build-Prozesse trotz vergleichbarer Anforderungen unterscheiden. Getrennt gepflegte Ausgangsstände für Web-, Cloud- und Desktop-Anwendungen erhöhen zudem den Wartungsaufwand und begünstigen auseinanderlaufende Technologie- und Dokumentationsstände.

Die zentrale Herausforderung besteht darin, Einheitlichkeit und Flexibilität zu verbinden. Das Template muss genügend gemeinsame Struktur für Standardisierung und Wiederverwendung vorgeben, darf ein Projekt jedoch nicht mit unnötigen Komponenten belasten. Untersucht wird daher, wie eine gemeinsame Basis, Projektprofile und optionale Erweiterungen diesen Zielkonflikt adressieren und wo Anpassungsbedarf verbleibt (kleiveist, 2026f, 2026g).

1.3 Zielsetzung

Ziel der Fallstudie ist die strukturierte Untersuchung des Projekts „Template-Projekte“. Analysiert werden die Architektur, die Aufteilung in gemeinsame und profilspezifische Bestandteile sowie die unterstützenden Entwicklungs- und Qualitätssicherungsprozesse (kleiveist, 2026f, 2026l).

Auf dieser Grundlage werden Wiederverwendbarkeit, Standardisierung und das Produktivitätspotenzial des Templates sowie seine Eignung für unterschiedliche Projektarten bewertet. Die Untersuchung stützt sich auf vorhandene Projektartefakte und Verifikationsnachweise. Daraus sollen Einsatzfelder und Auswahlkriterien für zukünftige Kundenprojekte sowie technische Grenzen und externe Voraussetzungen abgeleitet werden (kleiveist, 2026j). Eine universelle Eignung für sämtliche Softwareklassen oder eine produktspezifische Facharchitektur wird nicht beansprucht.

1.4 Fragestellungen

Aus der beschriebenen Problemstellung und Zielsetzung ergeben sich sechs zentrale Untersuchungsfragen. Sie strukturieren die Analyse des Templates und bilden den Bezugsrahmen für seine spätere Bewertung:

1. Welche fachlich-technischen, qualitativen und betrieblichen Anforderungen werden durch das Technologie-Template adressiert?
2. Wie unterstützen die Architektur und das Profilmodell die Bereitstellung unterschiedlicher Varianten für Web-, Cloud- und Desktop-Projekte?
3. Inwiefern fördern der gemeinsame technische Kern und die Single-Repository-Strategie die Wiederverwendung und Standardisierung zwischen Projekten?
4. Welches Potenzial bietet das Template, wiederkehrende Aufwände bei Projektinitialisierung, Entwicklungsworkflow, Qualitätssicherung und Bereitstellung zu reduzieren?
5. Für welche Projektarten und Kundenanforderungen stellt das Template eine geeignete technische Ausgangsbasis dar?
6. Welche technischen Grenzen, Wartungsaufwände, Risiken und externen Abhängigkeiten sind bei seinem Einsatz zu berücksichtigen?

Die Fragen sind miteinander verknüpft: Aussagen zu Produktivität und Eignung setzen zunächst eine nachvollziehbare Betrachtung der Anforderungen, der Architektur und der vorhandenen Verifikation voraus. Ihre Beantwortung wird daher nicht isoliert, sondern entlang der aufeinander aufbauenden Kapitel der Fallstudie vorgenommen.

1.5 Methodisches Vorgehen

Die Untersuchung ist als technische Fallstudie des implementierten Templates angelegt (Runeson & Höst, 2009). Ausgangspunkt ist eine Bestandsaufnahme des Master Repository. Dabei werden die Repository-Struktur, die enthaltenen Komponenten und die dokumentierten Verantwortungsgrenzen erfasst. Anschließend werden die Architekturentscheidungen und die Beziehungen zwischen Frontend, Backend, Desktop-Integration, gemeinsam genutzten Schnittstellen, Persistenz und Deployment betrachtet. Die Analyse beschreibt den vorhandenen Gegenstand und trennt belegbare Projekteigenschaften von Annahmen oder noch offenen Prüfpunkten (kleiveist, 2026f).

Darauf aufbauend wird das Profilmodell untersucht. Im Mittelpunkt stehen die Abgrenzung der Profile, die Aufteilung zwischen gemeinsamem Kern und profilspezifischen Bestandteilen sowie das Modell optionaler Capabilities. Die Betrachtung des Python-basierten Toolings ergänzt diese strukturelle Analyse um den praktischen Entwicklungsablauf. Hierzu werden insbesondere Projektgenerierung, Systemprüfung, Installation, Entwicklungsbetrieb, Tests, Builds und Release-Vorbereitung als zusammenhängender Workflow betrachtet. Die maSSgeblichen Profil- und Werkzeugbeschreibungen werden dabei als projektinterne Primärquellen herangezogen (kleiveist, 2026g, 2026l).

Für die Verifikation werden vorhandene automatisierte Tests, Build-Ergebnisse und dokumentierte Abnahmen den jeweils untersuchten Anforderungen und Architekturentscheidungen zugeordnet. Externe oder produktspezifische Prüfschritte werden davon abgegrenzt. Auf dieser Evidenzbasis erfolgt die Bewertung von Wiederverwendbarkeit, Standardisierung, Produktivitätspotenzial, Stärken und Grenzen. Abschließend werden daraus Kriterien für geeignete Einsatzfelder und für die Übertragung auf zukünftige Kundenprojekte abgeleitet (kleiveist, 2026j).

Abbildung 1 fasst diese aufeinander aufbauenden Untersuchungsschritte zusammen. Die vertikale Darstellung verdeutlicht, dass die Bewertung und der Transfer erst auf Grundlage der zuvor erhobenen und analysierten Projektinformationen erfolgen.

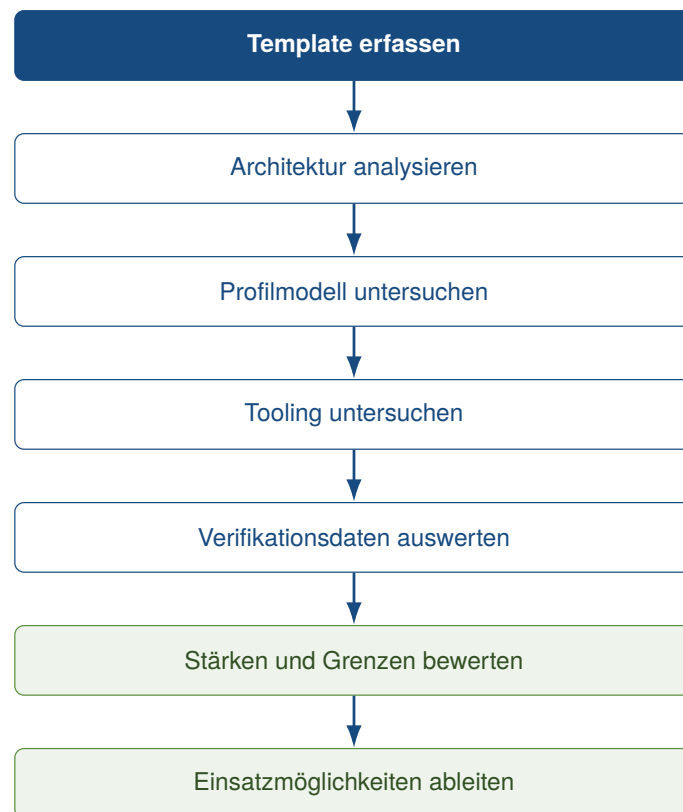


Abbildung 1: Methodische Schritte der Fallstudie

Quelle: Eigene Darstellung.

1.6 Aufbau der Fallstudie

Nach der Einleitung konkretisiert Abschnitt 2 die Anforderungen und die Abgrenzung des Technologie-Templates. Darauf folgt in Abschnitt 3 die Untersuchung seiner Gesamtarchitektur und der beteiligten technischen Komponenten. Abschnitt 4 betrachtet anschließend das Profil- und Feature-Modell einschließlich der optionalen Capabilities und der Single-Repository-Strategie. Das zentrale Projekt-Tooling sowie der Entwicklungs-, Build- und Release-Workflow werden in Abschnitt 5 untersucht.

Abschnitt 6 behandelt die Qualitätssicherung, die technische Verifikation und extern verbleibende Prüfungen. Auf dieser Grundlage bewertet Abschnitt 7 das Template hinsichtlich Produktivitätswirkung, Stärken, Grenzen und Wartungsaufwand. Abschnitt 8 überträgt die gewonnenen Erkenntnisse auf die Auswahl und Generierung zukünftiger Kundenprojekte. Die Arbeit endet mit der Schlussbetrachtung

in Abschnitt 9, in der die Ergebnisse gebündelt, die Fragestellungen beantwortet und offene Entwicklungsperspektiven eingeordnet werden.

2 Anforderungen an das Technologie-Template

2.1 Zielbild des Technologie-Templates

Das Template soll eine wiederverwendbare technische Ausgangsbasis für Web-, Cloud- und Desktop-Projekte bilden. Wiederverwendung setzt dabei geplante Gemeinsamkeiten und beherrschte Variabilität voraus (Clements & Northrop, 2001; Krueger, 1992). Projektstruktur, Entwicklungsworkflow, Tooling, Dokumentationsprinzipien, Konfigurationslogik, Tests und Schnittstellen sollen deshalb in einem gemeinsamen Kern zentral gepflegt werden.

Nicht jedes Projekt benötigt sämtliche Komponenten. Profile sollen passende Kombinationen des Kerns auswählen; optionale Capabilities wie PostgreSQL sollen kompatible Backend-Profile unabhängig vom Plattformprofil erweitern (kleiveist, 2026g). Bereits im Zielbild sind Frontend, Backend, Desktop-Schicht, Persistenz, Tooling und Deployment als getrennte Verantwortungsbereiche vorgesehen. Ob die Umsetzung diese Anforderungen erfüllt, wird erst in den nachfolgenden Architektur-, Verifikations- und Bewertungskapiteln geprüft.

2.2 Anforderungen an Wiederverwendbarkeit und Standardisierung

Einheitlich bedeutet nicht identisch: Die Baseline soll gemeinsame Strukturen und öffentliche Arbeitsabläufe vorgeben, zugleich aber kontrollierte Profilvarianten zulassen. Unabhängige Kopien sind zu vermeiden, weil sie Änderungen des gemeinsamen Fundaments mehrfach erforderlich machen und langfristig auseinanderlaufen können. Tabelle 1 fasst die daraus abgeleiteten, später überprüfbaren Anforderungen zusammen.

Tabelle 1: Zentrale Anforderungen an das Technologie-Template

ID	Anforderung	Operationalisiertes Ziel	spätere Prüfbasis
A-01	Baseline	Grundstruktur muss generierbar sein.	Projektgenerierung
A-02	Struktur	Verantwortlichkeiten liegen an konsistenten Stellen.	Repository-Struktur
A-03	Profile	Nur vorgesehene Komponenten werden aktiviert.	Profilmodell
A-04	Workflows	Prüfen, Initialisieren, Installieren, Starten, Testen und Bauen besitzen gemeinsame Einstiegspunkte.	Tooling
A-05	Wartbarkeit	Gemeinsame Bestandteile werden zentral gepflegt.	Repository-Strategie
A-06	Nachvollziehbarkeit	Änderungen und Konfiguration sind versioniert und dokumentiert.	Versionsstand, Dokumentation
A-07	Erweiterbarkeit	Optionale Fähigkeiten besitzen dokumentierte Abhängigkeiten.	Capability-Modell

Quelle: Eigene Darstellung.

Die Tabelle definiert Soll-Kriterien, keine festgestellten Eigenschaften. Ihre Erfüllung wird anhand der genannten Prüfbasis in den späteren Kapiteln beurteilt.

2.3 Anforderungen für Web-, Cloud- und Desktop-Anwendungen

Alle Varianten sollen eine gemeinsame Frontend-Basis, Projektstruktur, Konfigurationslogik sowie ein einheitliches Test- und Dokumentationsmodell nutzen. Browserprojekte benötigen keine native Desktop-Schicht und nur bei Bedarf ein Backend. Cloudfähige Varianten erfordern eine serverseitige API, konfigurierbare Deployment-Grenzen sowie Health- und Readiness-Prüfbarkeit. Desktopvarianten sollen dasselbe Frontend in einer nativen Laufzeit verwenden und Weblogik von plattformspezifischer Integration trennen. Kombinierte Varianten verbinden Desktop-Client, Backend und gegebenenfalls Browserzugriff.

Tabelle 2: Anforderungsmatrix der unterstützten Projektarten

Anforderung	Web	Cloud	Desktop lokal	Desktop + Cloud
Frontend	erforderlich	erforderlich	erforderlich	erforderlich
Backend / API	optional	erforderlich	nicht grundsätzlich	erforderlich
Desktop-Laufzeit	nein	nein	erforderlich	erforderlich
Deployment-Grenze	optional	erforderlich	nein	erforderlich
Persistenz	optional	optional	produktspezifisch	optional

Quelle: Eigene Darstellung auf Grundlage von kleiveist (2026g).

Die Matrix formuliert Anforderungen, keine Prüfergebnisse. PostgreSQL ist dabei ausschließlich eine optionale Capability für backendfähige Profile und kein eigenes Plattformprofil (kleiveist, 2026g). Öffentliche Client-Konfiguration und serverseitige Werte einschließlich Geheimnissen sind voneinander zu trennen; eine bestimmte Cloud-Plattform wird nicht vorausgesetzt.

2.4 Qualitäts- und Wartbarkeitsanforderungen

Die Qualitätsanforderungen orientieren sich an überprüfbaren Merkmalen des Produktqualitätsmodells nach ISO/IEC 25010 (ISO/IEC, 2023). Komponenten und Profile sollen klare Verantwortlichkeiten besitzen, getrennt testbar und durch dokumentierte Architektur-, Profil- und Workflowentscheidungen wartbar sein. Neue optionale Fähigkeiten sowie Technologieaktualisierungen sollen ohne Duplizierung der gesamten Baseline möglich bleiben; daraus folgt keine Annahme, dass Aktualisierungen automatisch risikofrei sind.

Gleiche Eingaben und Konfigurationen sollen nachvollziehbare Generierungs- und Build-Ergebnisse ermöglichen. Reproduzierbare Builds erhöhen die Prüfbarkeit der Beziehung zwischen Quellstand und Artefakt (Lamb & Zacchioli, 2022). Ferner sollen relevante Zielartefakte kontrolliert erzeugbar, Konfigurationsquellen und Prioritäten verständlich definiert und serverseitige Geheimnisse von Frontend- und Client-Konfiguration getrennt sein. Die tatsächliche Erfüllung wird in Kapitel 6 untersucht.

2.5 Abgrenzung

Das Zielbild ist keine universelle Softwareplattform. Im Mittelpunkt stehen wiederkehrende Web-, Backend-, Desktop- und Full-Stack-Business-Anwendungen. Geschäftsregeln, Domänen- und Kundendatenmodelle sowie produktspezifische Authentifizierungs-, Rollen- und Berechtigungskonzepte gehören nicht zur technischen Baseline. Ebenso wird kein bestimmter Cloud-Provider vorausgesetzt (kleiveist, 2026f).

Stark plattformspezifische native Anwendungen, Spiele und hochspezialisierte Echtzeitsysteme zählen nicht automatisch zum Zielbereich. Dieses Kapitel definiert lediglich Anforderungen und Grenzen; eine Eignungszusage erfolgt erst auf Grundlage der späteren Verifikation und Bewertung.

3 Architektur des Technologie-Templates

3.1 Gesamtarchitektur

Die untersuchte Baseline ist ein Master-Repository mit gemeinsam gepflegtem Kern und deklarativen Varianten, kein zur Laufzeit zusammenhängender Monolith. Profile wählen wiederverwendbare Features aus; optionale Capabilities ergänzen nur compatible Profile. Dieses Vorgehen überträgt das Prinzip beherrschter Variabilität aus Software-Produktlinien auf die Projektstruktur und adressiert damit die in Kapitel 2 geforderte zentrale Wartbarkeit ohne getrennte Template-Kopien (Clements & Northrop, 2001; kleiveist, 2026g).

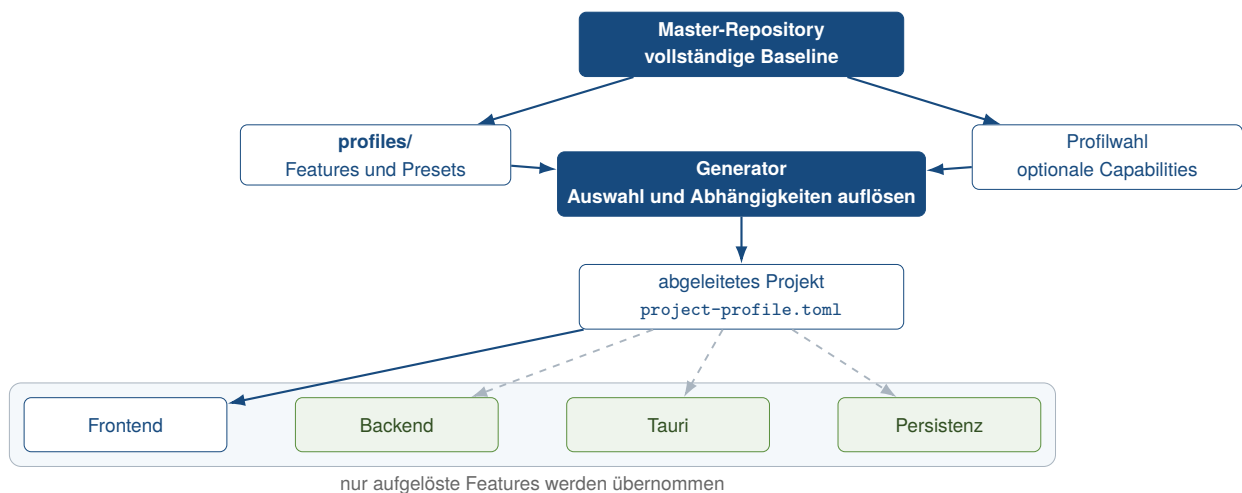


Abbildung 2: Gesamtarchitektur des Technologie-Templates

Quelle: Eigene Darstellung auf Grundlage von kleiveist (2026f) und kleiveist (2026g).

Abbildung 2 trennt deshalb die vollständige Baseline von einem abgeleiteten Projekt. Der Generator löst Profil und Capabilities auf, übernimmt nur die zugehörigen Pfade und hält das Ergebnis in `project-profile.toml` fest. Frontend, Backend, Tauri und Persistenz bleiben eigenständige, teilweise optionale Bausteine (kleiveist, 2026f).

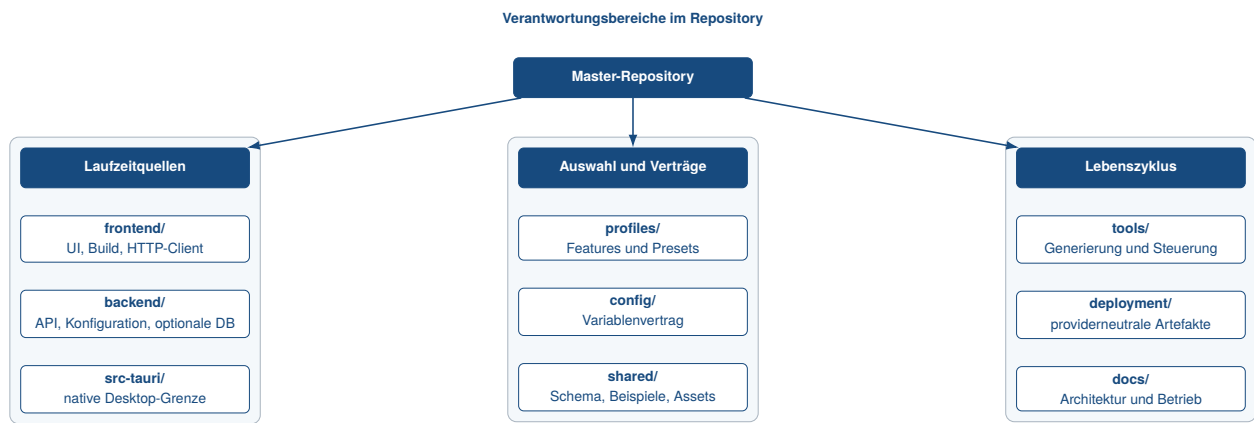


Abbildung 3: Komponenten und Verantwortungsgrenzen im Master Repository

Quelle: Eigene Darstellung auf Grundlage von kleiveist (2026f).

Abbildung 3 ordnet die Verantwortungen konkreten Verzeichnissen zu. `frontend/` enthält Darstellung und HTTP-Client, `backend/` die serverseitige API, `src-tauri/` die native Grenze. `shared/` hält frameworkneutrale Artefakte; `profiles/` und `config/` beschreiben Struktur beziehungsweise Laufzeitwerte. Tooling und Deployment steuern diese Komponenten, gehören aber nicht zu deren Produktlaufzeit.

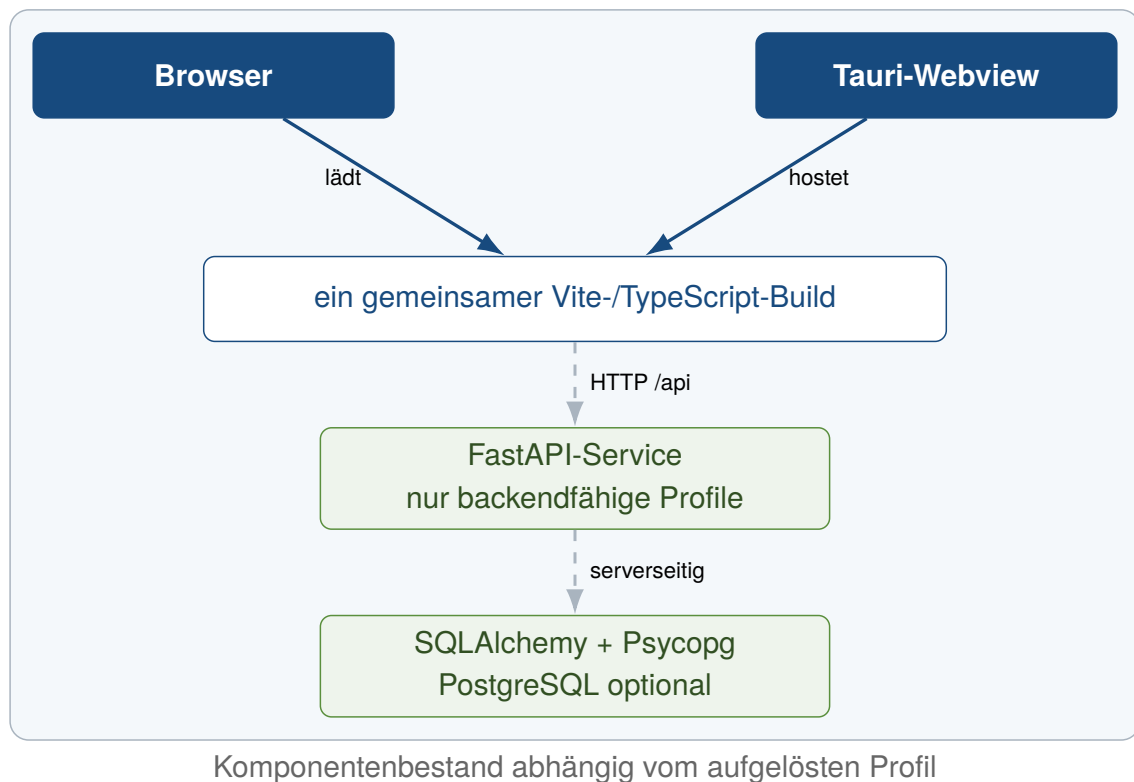


Abbildung 4: Profilabhängige Laufzeitarchitektur

Quelle: Eigene Darstellung auf Grundlage von kleiveist (2026f) und kleiveist (2026g).

Zur Laufzeit wird derselbe Vite-Build entweder vom Browser oder in einer Tauri-Webview geladen (Abbildung 4). Nur backendfähige Profile ergänzen die Kommunikation über die HTTP-Grenze zu FastAPI; ausschliesslich das Backend darf eine aktivierte Persistenzschicht ansprechen. Damit bleiben Client, Server und Datenhaltung getrennt test- und erweiterbar, ohne dass jedes Profil alle Laufzeiteinheiten

enthalten muss.

3.2 Frontend mit Vite und TypeScript

Das Frontend bildet die gemeinsame Präsentationsschicht aller fünf Profile. Vite stellt im Entwicklungsbetrieb den Modulserver bereit und erzeugt mit `vite build` statische Artefakte; der vorgeschaltete TypeScript-Lauf prüft den strikt konfigurierten Quellcode ohne JavaScript-Ausgabe. Vitest deckt den HTTP-Client ab. Diese Aufgabenteilung entspricht den dokumentierten Rollen von Vite als Entwicklungs- und Build-Werkzeug sowie TypeScript als statischer Typprüfung (Microsoft, 2026; VoidZero Inc. and Vite Contributors, 2026).

`frontend/src/main.ts` liest das generierte Profilmodul und zeigt die Backend-Integration nur bei aktiviertem Feature. Alle Aufrufe liegen in `frontend/src/api/`; `backend.ts` verlangt dann eine öffentliche `VITE_API_BASE_URL` und ruft `/api/health` auf. Dieselben Quellen werden im Browser und in Tauri verwendet. Serverlogik, Geheimnisse und direkter Datenbankzugriff bleiben außerhalb dieser Grenze; Web-only und Desktop-local benötigen folglich kein Backend (kleiveist, 2026f, 2026g).

Die im untersuchten Commit deklarierten Bereiche und aufgelösten Versionen sind in Tabelle 3 zusammengefasst.

Tabelle 3: Deklarierter und aufgelöster Technologie-Stack im untersuchten Commit

Technologie	Rolle im Template	Version / Quelle	Projektbeleg
Vite	Entwicklungsserver und Frontend-Build	Bereich ab 6.0.7; Lock: 6.4.2	<code>frontend/package.json</code> , <code>frontend/package-lock.json</code>
TypeScript	statische Prüfung des Frontends	Bereich ab 5.7.3; Lock: 5.9.3	<code>frontend/package.json</code> , <code>frontend/tsconfig.json</code>
FastAPI	HTTP-API und Validierung	≥ 0.111 , < 1 ; Produktion: 0.111.0	<code>backend/requirements.txt</code> , <code>backend/requirements-production.txt</code>
Tauri	Desktop-Webview und Packaging-Grenze	Major 2; Lock: 2.11.5	<code>src-tauri/Cargo.toml</code> , <code>src-tauri/Cargo.lock</code>
Rust	native Kompositionsschicht	mindestens 1.77; Edition 2021	<code>src-tauri/Cargo.toml</code>
PostgreSQL	optionale relationale Laufzeiteinheit	Container: 16.4-alpine3.20	<code>deployment/compose.yaml</code>
SQLAlchemy	Engine-, Session- und Modellbasis	≥ 2 , < 3 ; Produktion: 2.0.36	<code>backend/requirements-database*.txt</code>
Alembic	explizite Schemamigrationen	≥ 1.13 , < 2 ; Produktion: 1.14.0	<code>backend/requirements-database*.txt</code>
Psycopg	PostgreSQL-Treiber	≥ 3.2 , < 4 ; Produktion: 3.2.3	<code>backend/requirements-postgres*.txt</code>

Quelle: Eigene Darstellung auf Grundlage des geprüften Projektstands (kleiveist, 2026k).

3.3 Backend mit FastAPI

Das Backend ist eine separate HTTP-Schicht der Profile Web-cloud, Desktop-cloud und Full-platform. `backend/app/main.py` erzeugt die FastAPI-Anwendung, lädt typisierte Einstellungen, richtet CORS ein und bindet den Router unter `/api` ein. Der derzeitige Router enthält ausschließlich `/health` und `/ready`; produktspezifische Dienste, Fachmodelle und Authentifizierung sind bewusst nicht vorgegeben. FastAPI stellt hierfür die HTTP-Routen, Validierung und OpenAPI-Beschreibung bereit (kleiveist, 2026f; Ramírez, 2026).

Liveness meldet nur den Prozesszustand. Readiness prüft bei aktivierter Datenbankfähigkeit zusätz-

lich mit `SELECT 1` und antwortet bei fehlender Konfiguration oder Verbindung mit HTTP 503, ohne interne Fehler offenzulegen. Tests adressieren Router, Einstellungen und diese Abhängigkeit getrennt. Die Architektur hält Handler klein; erst ein abgeleitetes Produkt ergänzt Service- oder Domänenmodule.

Auch im Deployment bleiben die Einheiten getrennt (Abbildung 5). Der Vite-Build kann statisch bereitgestellt werden, während FastAPI in einem eigenen, nicht privilegierten Container läuft. Compose simuliert diese providerneutrale Grenze lokal; PostgreSQL wird nur über ein explizites Profil ergänzt (kleiveist, 2026e).

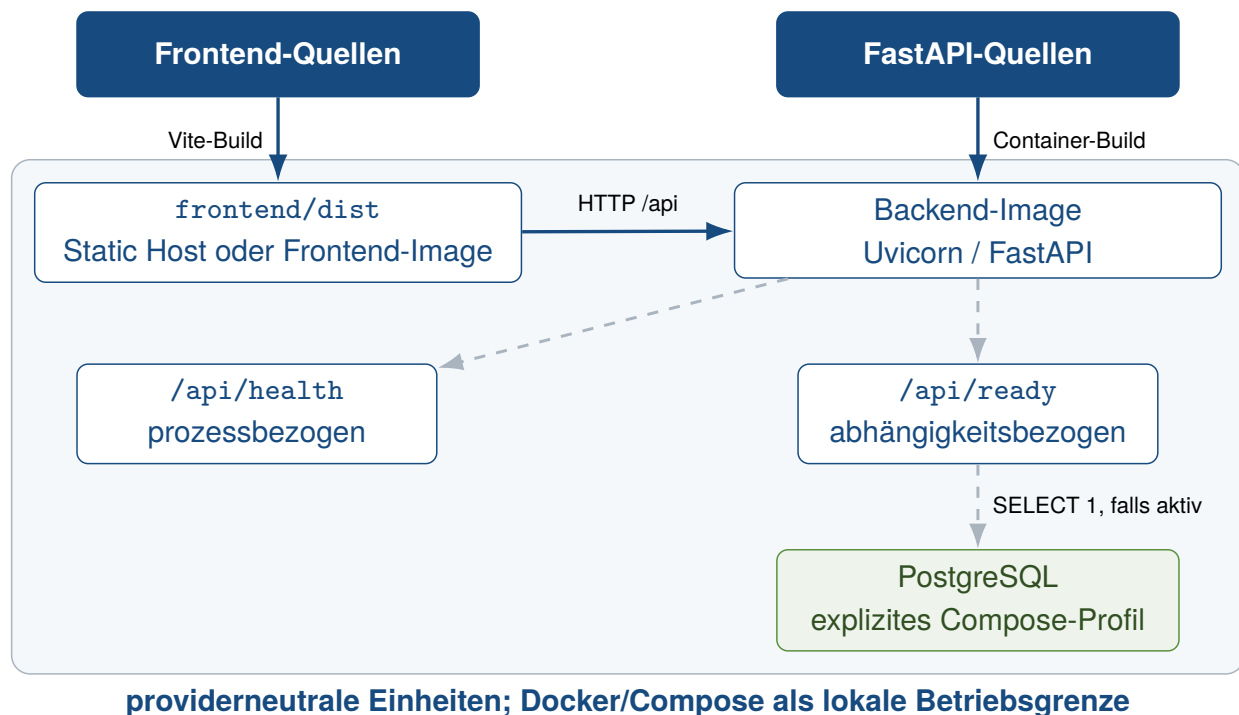


Abbildung 5: Providerneutrale Deployment-Architektur

Quelle: Eigene Darstellung auf Grundlage von kleiveist (2026e).

3.4 Desktop-Integration mit Tauri und Rust

Tauri bildet eine eigene native Grenze, ohne das Frontend zu duplizieren. `src-tauri/tauri.conf.json` startet für die Entwicklung ausschliesslich den Vite-Server und verwendet für Builds `frontend/dist`. Der Rust-Einstieg erstellt lediglich die Tauri-Anwendung; native Fachkommandos oder ein eingebetteter FastAPI-Sidecar sind im Template nicht implementiert. Dies nutzt Tauris Fähigkeit, Web-Frontends in einer nativen Webview zu hosten, während Rust die native Kompositionsgrenze bereitstellt (Klabnik et al., 2026; kleiveist, 2026f; Tauri Contributors, 2026b).

Die Standard-Capability gewährt nur `core:default`; eine Content Security Policy begrenzt lokale Ressourcen und die dokumentierten Entwicklungsendpunkte. Weitere native Berechtigungen müssen produktspezifisch und möglichst eng ergänzt werden (Tauri Contributors, 2026a). Im Profil `Desktop-local` bleibt Funktionalität lokal im Frontend oder in später ergänzten Tauri-Kommandos. `Desktop-cloud` und `Full-platform` verwenden dagegen die öffentliche FastAPI-URL. Packaging, Signierung und die Entscheidung zwischen Remote-API, Sidecar oder lokalen Kommandos sind Produktentscheidungen und nicht Teil dieser Baseline.

3.5 Shared Contracts und Schnittstellen

`shared/` gehört zum Kern jedes generierten Projekts und enthält statische Assets, ein JSON-Schema nach Draft 2020-12 sowie je ein gültiges und ungültiges Beispiel. Die Schema-Tests des Toolings validieren diese Beispiele; die Ablage bleibt damit serialisierbar und frameworkneutral. Sie definiert jedoch noch keine produktbezogene API und enthält keinen ausführbaren Frameworkcode (kleiveist, 2026f, 2026k).

Der Implementierungsabgleich begrenzt diese Aussage: Weder Frontend noch Backend importieren das vorhandene Eingabeschema zur Laufzeit. Auch der Health-Vertrag ist nicht daraus generiert, sondern als TypeScript-Interface und FastAPI-Antwort manuell gespiegelt. Eine automatische Codegenerierung oder Synchronisierung ist nicht vorhanden. Vertragsänderungen erfordern daher derzeit eine bewusste Anpassung beider Seiten und ihrer Konsumententests. Diese Abweichung zwischen der dokumentierten Zielrolle gemeinsam konsumierter Verträge und der aktuellen Nutzung ist eine offene Erweiterungsgrenze, kein Nachweis einer bereits automatisierten Schnittstellenkopplung.

3.6 Konfigurationsmodell

Projektstruktur und Laufzeitkonfiguration sind getrennte Achsen: `project-profile.toml` bestimmt die vorhandenen Features, `config/environment.toml` den Variablenvertrag. Der zentrale Resolver berücksichtigt nur Variablen der aktiven Features und wendet in absteigender Priorität CLI-Override, Prozessumgebung, lokale `.env` und Vertragsdefault an. Ableitungen wie die lokale API-URL bleiben als Quelle nachvollziehbar (kleiveist, 2026i).

Abbildung 6 zeigt die Adapter- und Sicherheitsgrenzen. Vite bettet ausschließlich die explizit freigegebene `VITE_API_BASE_URL` ein. FastAPI verarbeitet serverseitige Werte typisiert; `DATABASE_URL` bleibt ein maskiertes Secret. Tooling gibt nur relevante Werte an Kindprozesse weiter, und die Tauri-Umgebung entfernt bekannte Secret-Namen. Ungültige Hosts, Ports, Origins oder erforderliche fehlende Werte führen zu einem kontrollierten Abbruch, ohne den geheimen Eingabewert in der Fehlermeldung auszugeben.

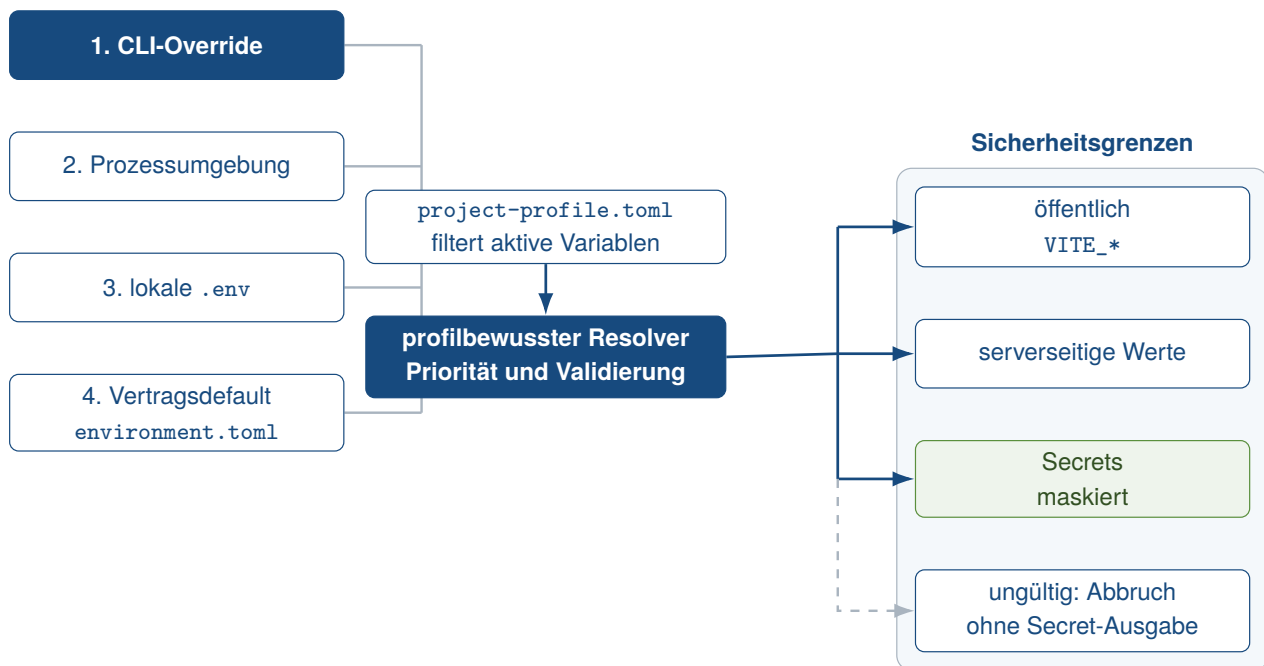


Abbildung 6: Konfigurationspriorität und Sicherheitsgrenzen

Quelle: Eigene Darstellung auf Grundlage von kleiveist (2026i).

3.7 Persistenz mit PostgreSQL, SQLAlchemy und Alembic

PostgreSQL ist keine Pflichtkomponente und kein eigenes Profil. Die wählbare Capability `postgres` setzt `database` voraus, das wiederum ein Backend benötigt; `Web-only` und `Desktop-local` werden deshalb abgewiesen. Die generische Datenbankfähigkeit enthält SQLAlchemy-Engine, Sessions und Alembic, während die PostgreSQL-Erweiterung den Psycopg-3-Treiber, URL-Vorgaben und Integrationstests ergänzt. PostgreSQL selbst bleibt eine getrennte Laufzeiteinheit (kleiveist, 2026d; PostgreSQL Global Development Group, 2026).

`backend/app/db/engine.py` erzeugt die Engine erst beim ersten Zugriff und aktiviert `pool_pre_ping`; Sessions werden über eine Generator-Dependency geschlossen. Die leere deklarative Base ist nur ein Erweiterungspunkt für spätere Fachmodelle. Das entspricht der von SQLAlchemy dokumentierten Trennung von Engine, Pool, Dialekt und Session sowie dem verzögerten Aufbau der ersten Verbindung (SQLAlchemy Authors and Contributors, 2026).

Alembic verwendet dieselbe Metadatenbasis für Online- und Offline-Migrationen. Da das Template keine Fachmodelle enthält, existiert noch keine initiale Revision; weder `create_all()` noch automatische Migrationen beim FastAPI-Start sind implementiert. Schemaänderungen erfolgen ausschließlich über explizite Alembic-Kommandos, deren Umgebung und Revisionsablauf die offizielle Dokumentation beschreibt (Bayer, 2026; kleiveist, 2026d). `/api/health` bleibt datenbankunabhängig, während `/api/ready` bei aktivierter Datenbankfähigkeit die Erreichbarkeit prüft.

4 Projektprofile und Feature-Modell

4.1 Grundprinzip des Profilmodells

Wie in Kapitel 3 beschrieben, bildet ein Master-Repository die gemeinsame technische Ausgangsbasis. Das Modell weist damit Merkmale featurebasierter Variabilitätsverwaltung und systematischer Softwarewiederverwendung auf, ohne allein deshalb eine vollständige Software-Produktlinie zu konstituieren (Clements & Northrop, 2001; Krueger, 1992). Der Katalog `profiles/features.toml` trennt vier Begriffe: Der *Core* umfasst die in jedes Produktprojekt kopierten Pfade `README.md`, `VERSION`, `.gitignore`, `config/`, `docs/`, `profiles/`, `shared/` und `tools/`. Ein *Feature* ordnet einer technischen Fähigkeit eigene Scaffold-Pfade und gegebenenfalls Abhängigkeiten zu. Ein *Profil* ist dagegen ein benanntes Preset zulässiger Features. Eine *optionale Capability* erweitert dieses Preset nach der Profilwahl; sie ist weder ein Plattformprofil noch automatisch Bestandteil eines Profils (kleiveist, 2026g, 2026k).

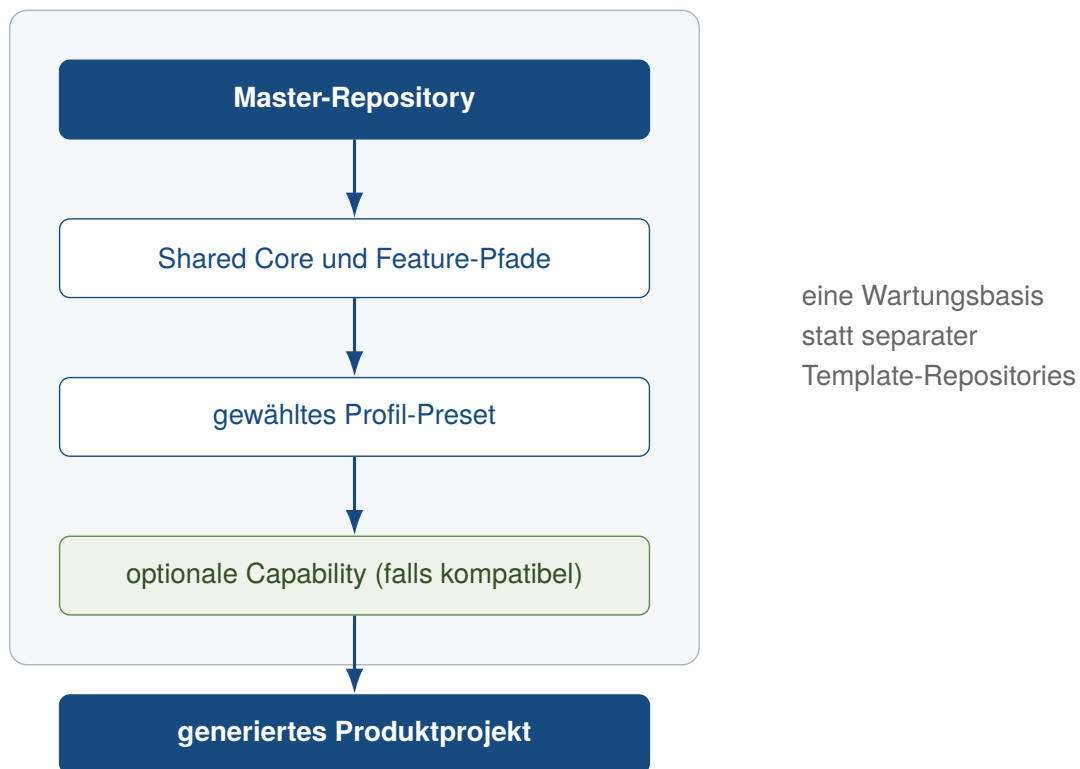


Abbildung 7: Ableitung eines Produktprojekts aus einer gemeinsamen Template-Basis

Abbildung 7 fasst diese Ebenen zusammen. Sie abstrahiert von den Laufzeitkomponenten und hebt stattdessen hervor, dass Core, Feature-Katalog und Presets in einer Wartungsbasis liegen. Der Generator löst das gewählte Preset und kompatible Capabilities auf, bildet aus Core- und Feature-Pfaden einen Scaffold-Plan und kopiert diesen in ein neues Produktprojekt. Nicht aktivierte Verzeichnisse werden somit überwiegend gar nicht ausgewählt; bei Webprofilen entfernt die Nachbearbeitung zusätzlich die Tauri-CLI aus den kopierten Frontend-Abhängigkeiten. Optional können Name, Slug und bei Tauri die Anwendungs-ID angepasst werden (kleiveist, 2026g, 2026l).

Die Auswahl in Abbildung 8 berücksichtigt, dass die gegenwärtigen Presets Backend und Cloud gemeinsam führen. Bei Desktopprojekten trennt der zusätzliche Browser-Produktkanal semantisch

full-platform von desktop-cloud; erst nach der Profilwahl wird eine Capability geprüft. Die anschließende Abhängigkeitsauflösung verhindert, dass dadurch stillschweigend ein fehlendes Plattformfeature ergänzt wird.

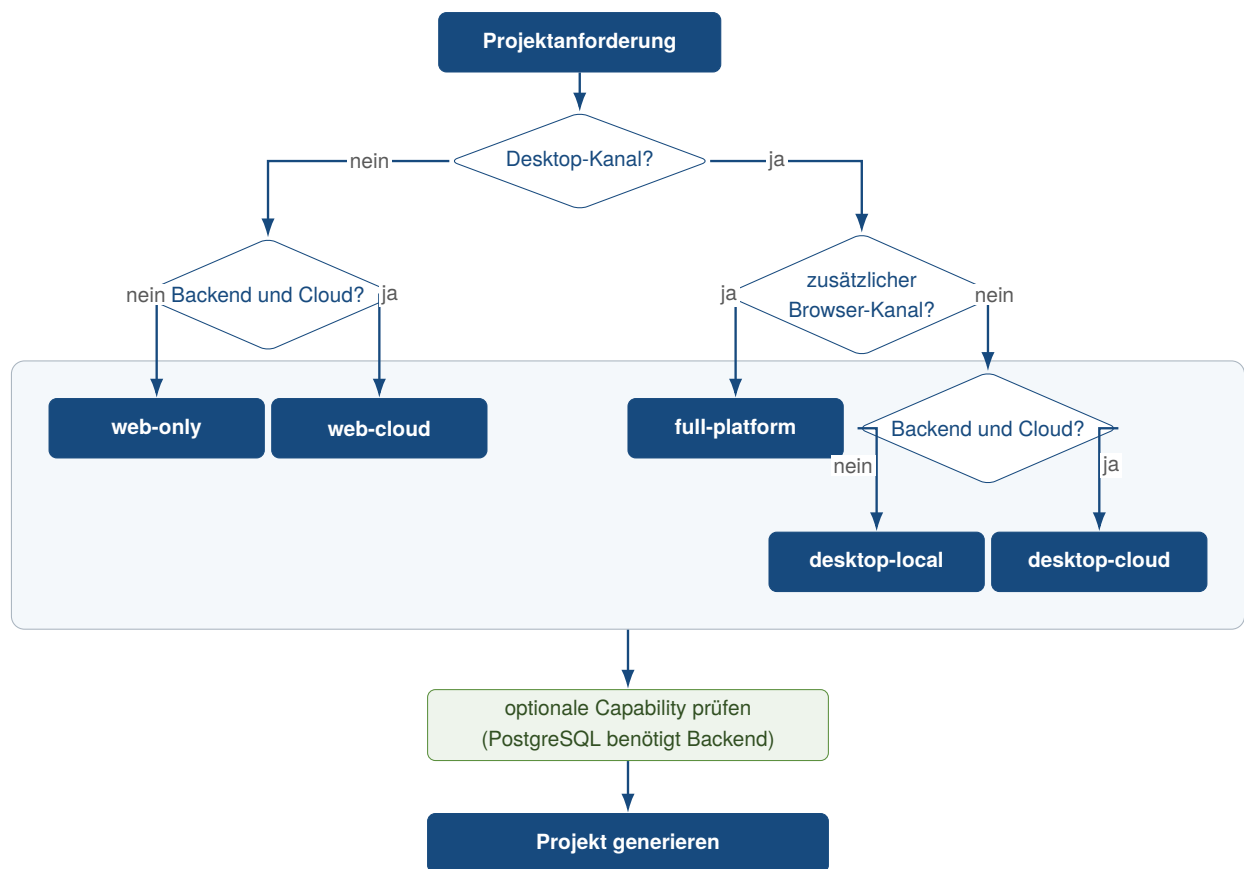


Abbildung 8: Entscheidungsfluss für Profil und anschließende Capability-Auswahl

Das aktive Manifest heit `project-profile.toml`. Es wird fr jedes generierte Projekt erzeugt. `schema_version` hlt die Schema-Version fest. `id`, `name` und `description` beschreiben das Profil. `optional_features` erfasst die angeforderten Erweiterungen; `features` enthlt die vollstndige Auflsung. Das Tooling ldt und validiert das Manifest. Dadurch behandelt es nur aktive Dienste und Workflows. Im Master nennt das Manifest `full-platform` als Profil. Zustzlich aktiviert es `postgres`; das Preset selbst bleibt davon unberhrt. Profilstruktur und Laufzeitwerte sind getrennte Achsen. Development-, Test- und Produktionswerte, Ports, URLs und Geheimnisse stammen aus dem in Abschnitt 3.6 beschriebenen Konfigurationsmodell (kleiveist, 2026i, 2026k).

Abbildung 9 ermglicht den schnellen strukturellen Vergleich. Tabelle 4 ergnzt dazu die exakt deklarierten Feature-IDs und die daraus ausgewhlten Laufzeitverzeichnisse. Insbesondere ist PostgreSQL bei keinem der fnf Presets voreingestellt.

Profil	Frontend	FastAPI	Tauri	Cloud	PostgreSQL
web-only	enthalten	–	–	–	inkompatibel
web-cloud	enthalten	enthalten	–	enthalten	optional
desktop-local	enthalten	–	enthalten	–	inkompatibel
desktop-cloud	enthalten	enthalten	enthalten	enthalten	optional
full-platform	enthalten	enthalten	enthalten	enthalten	optional

Abbildung 9: Strukturelle Feature-Zuordnung der fünf Profil-Presets

Quelle: Eigene Darstellung auf Grundlage des Feature-Katalogs und der Profildefinitionen im geprüften Projektstand (kleiveist, 2026g, 2026k).

Tabelle 4: Deklarierte Basisfeatures und resultierende Laufzeitverzeichnisse

Profil	Basisfeatures	Laufzeitverzeichnisse	PostgreSQL
web-only	frontend	frontend/	inkompatibel
web-cloud	frontend, backend, cloud	frontend/, backend/, deployment/	optional
desktop-local	frontend, tauri	frontend/, src-tauri/	inkompatibel
desktop-cloud	frontend, backend, tauri, cloud	frontend/, backend/, src-tauri/, deployment/	optional
full-platform	frontend, backend, tauri, cloud	frontend/, backend/, src-tauri/, deployment/	optional

4.2 Profil Web-only

web-only aktiviert als einziges Plattformfeature frontend. Zum Core kommt daher frontend/. Nicht in den Scaffold gelangen backend/, src-tauri/, deployment/ sowie Datenbank- und PostgreSQL-Abhängigkeiten. Der Generator entfernt außerdem Tauri-Skript und -CLI aus package.json und Lockfile. Das Laufzeitmodell ist damit ein im Browser geladenes Vite-Frontend ohne FastAPI- oder native Desktop-Schicht; auch das generierte Frontend-Profil aktiviert keinen Backend-Statusaufruf. Gegenüber web-cloud fehlen folglich API und Cloud-/Deployment-Feature (kleiveist, 2026g).

4.3 Profil Web-cloud

web-cloud deklariert frontend, backend und cloud. Der Scaffold enthält damit Vite-Frontend, den für das Basisbackend ausgewählten FastAPI-Code sowie deployment/ und .dockerignore; src-tauri/ und dessen npm-Abhängigkeiten werden nicht übernommen. Das Frontend kommuniziert über die öffentliche API-URL mit dem getrennten Backend. cloud kennzeichnet Deployment-Dateien ohne Bindung an einen konkreten Anbieter (kleiveist, 2026e, 2026g).

Ohne weitere Auswahl fehlen Datenbankmodul, Alembic sowie die Datenbank- und Psycpg-Dateien. PostgreSQL ist deshalb eine kompatible, aber separat anzufordernde Capability; `web-cloud` ist nicht gleichbedeutend mit `web-cloud + postgres`.

4.4 Profil Desktop-local

`desktop-local` kombiniert `frontend` und `tauri`; die Abhängigkeit von Tauri zum Frontend ist damit explizit erfüllt. Derselbe Frontend-Build läuft in der Tauri-Webview, während `src-tauri/` die Rust-basierte Desktop-Schicht bereitstellt. Ein FastAPI-Prozess ist hierfür nicht erforderlich: `backend/`, `deployment/` und die Datenbankpfade werden nicht generiert (kleiveist, 2026g).

Das Suffix *local* bezeichnet somit die fehlende Cloud-API. Eine lokale SQL-Datenbank ist nicht eingebaut. Da `database` ein Backend voraussetzt, ist auch die PostgreSQL-Capability mit diesem Profil nicht kompatibel.

4.5 Profil Desktop-cloud

`desktop-cloud` aktiviert `frontend`, `backend`, `tauri` und `cloud`. Es ergänzt `desktop-local` damit um FastAPI und die Deployment-Grenze: Das gemeinsame Frontend wird in der Tauri-Webview ausgeführt und greift über die konfigurierte öffentliche URL auf das getrennte Backend zu. Datenbankinfrastruktur bleibt ohne Capability ausgeschlossen; PostgreSQL kann wegen des vorhandenen Backends ergänzt werden (kleiveist, 2026e, 2026g).

Die Basisfeatures und Feature-Pfade sind identisch zu `full-platform`. Die Profil-ID dokumentiert jedoch den Desktop-Client als vorgesehenen Produktkanal, während das folgende Profil Browser und Desktop gemeinsam adressiert. Ein zusätzlicher technischer Baustein entsteht aus dieser semantischen Abgrenzung nicht.

4.6 Profil Full-platform

Auch `full-platform` deklariert `frontend`, `backend`, `tauri` und `cloud`. Semantisch steht das Preset für zwei Produktkanäle aus derselben Frontend-Basis. Browser-Build und Tauri-Desktopanwendung greifen auf dasselbe FastAPI-Backend zu. Im Scaffold unterscheidet sich das Profil von `desktop-cloud` nur durch ID, Name und Beschreibung im Manifest beziehungsweise im generierten Frontend-Profil; der Scaffold-Plan der Basisfeatures ist gleich (kleiveist, 2026g, 2026k).

Die Bezeichnung *full* schaltet keine Persistenz hinzu. Erst die optionale Auswahl von `postgres` ergänzt `database` und `postgres`; ohne sie bleibt auch dieses Profil datenbankfrei.

4.7 Optionale Capabilities

Der Katalog enthält sechs Features. Vier Abhängigkeiten sind implementiert: `tauri` benötigt `frontend`, `cloud` und `database` benötigen jeweils `backend`, und `postgres` benötigt `database`. Abbildung 10 zeigt nur diese gerichteten Regeln; sie bildet weder Laufzeitkommunikation noch mögliche zukünftige Features ab (kleiveist, 2026d, 2026g).

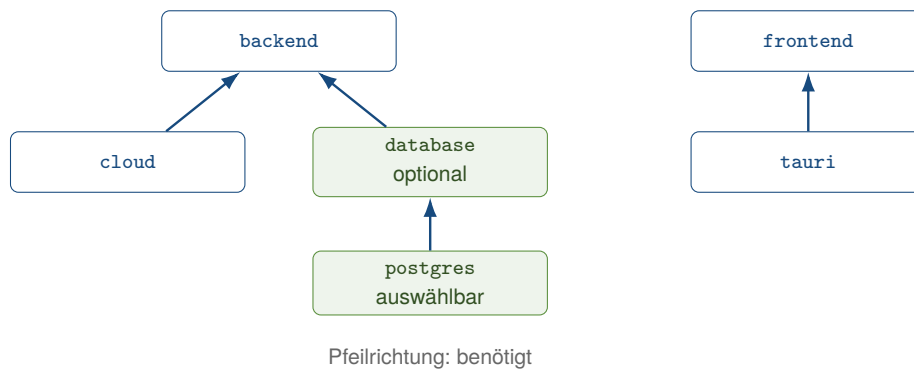


Abbildung 10: Implementierte Abhängigkeiten des Feature-Katalogs

Als benutzerseitig vorgesehene Capability ist gegenwärtig allein `postgres` mit `optional = true` und `selectable = true` markiert. Für ein backendfähiges Profil löst die Auswahl zunächst die ebenfalls optionale, aber nicht als auswählbar markierte Infrastruktur `database` auf. So entsteht beispielsweise `web-cloud + postgres` aus `frontend`, `backend`, `cloud`, `database` und `postgres`. Die Erweiterung fügt SQLAlchemy-/Alembic-Code, Datenbanktests und zugehörige Anforderungsdateien sowie Psycopy-Dateien und PostgreSQL-Integrationstests hinzu. Das gemeinsame Cloud-Compose-Dokument enthält bereits ohne diese Auswahl einen standardmäßigen inaktiven Dienst unter dem Compose-Profil `postgres`; die Capability liefert daher nicht erst dessen Deklaration, sondern die zugehörige Backend-Infrastruktur und Abhängigkeiten (kleiveist, 2026e, 2026k).

Die Auflösung ergänzt ausschließlich fehlende *optionale* Voraussetzungen. Ein nicht optionales Plattformfeature wird nicht stillschweigend aktiviert. Deshalb lehnt das Tooling `web-only + postgres` und `desktop-local + postgres` ab: beiden fehlt `backend`. Neue Capabilities folgen demselben dokumentierten Weg über Katalogeintrag, Abhängigkeiten, eigene Pfade und Tests; als implementiert gelten sie dadurch noch nicht.

Der Vergleich von Dokumentation und Code zeigt zwei Abweichungen. Die interaktive Auswahl bietet entsprechend `selectable` nur PostgreSQL an; bei einer direkten `-with database`-Angabe prüft der Resolver jedoch nur `optional` und akzeptiert die generische Datenbankfähigkeit für Backendprofile. Ferner liegen die Einträge für die generierte `.env.example` nicht in `features.toml`, wie die Profildokumentation allgemein formuliert, sondern werden anhand von `config/environment.toml` und den aufgelösten Features ausgewählt. Diese Unterschiede ändern die fünf Presets nicht, begrenzen aber die deklarierte Trennung zwischen interner und auswählbarer Capability.

4.8 Single-Repository-Strategie

Die fünf Profile sind keine langfristig gepflegten Template-Forks. Im Master-Repository liegen Core, vollständige Feature-Implementierungen, Profildefinitionen und Generator gemeinsam; `features.toml` und die fünf Preset-Dateien bilden dabei die deklarative Quelle für Pfade und Abhängigkeiten. Damit existiert eine technische Wartungsbasis, aus der der Generator eigenständige Produktprojekte ableitet (kleiveist, 2026g, 2026k).

Master-Repository und Produktprojekt sind folglich zu unterscheiden: Das Master enthält auch optionale Implementierungen und aktiviert in seinem eigenen Manifest PostgreSQL, damit diese Basis wartbar bleibt. Das generierte Projekt erhält Core plus die Pfade seines aufgelösten Profils, ein neu-

es `project-profile.toml`, das generierte Frontend-Profil und eine featureabhängige `.env.example`. Es bleibt anschließend ein separates Projektverzeichnis; der Generator verändert das Master nicht. Die Struktur reduziert konzeptionell die Zahl unabhängiger Template-Baselines und adressiert damit die in Kapitel 2 formulierte kontrollierte Variabilität. Eine vergleichende Bewertung dieser Konsequenz erfolgt erst im Bewertungskapitel.

5 Tooling und Entwicklungsworkflow

5.1 Rolle von `control.py`

`tools/control.py` ist der öffentliche Einstiegspunkt für die projektweiten Arbeitsabläufe. Der mit `argparse` aufgebaute Parser stellt im untersuchten Stand die Befehle `init`, `doctor`, `install`, `console`, `run`, `stop`, `test`, `build`, `config`, `db`, `tauri`, `container`, `version`, `release` und `docs` bereit. Gruppenbefehle zeigen ohne Aktion ihre nächste Befehlsebene; ebenso liefern ein leerer Aufruf und `-help` ein Orientierungsmodell, ohne einen Workflow auszuführen (kleiveist, 2026a, 2026l).

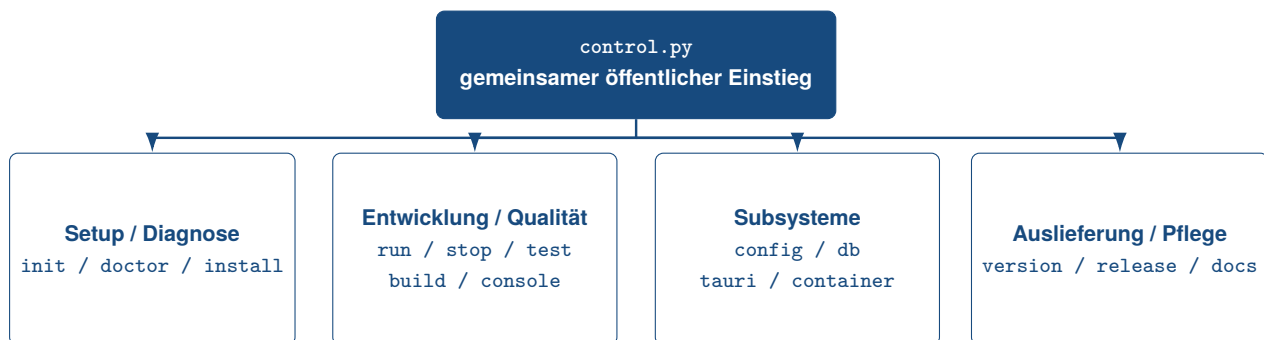


Abbildung 11: Befehlsgruppen des zentralen Projekt-Toolings

Quelle: Eigene Darstellung auf Grundlage von kleiveist (2026a) und kleiveist (2026l).

Die gemeinsame Oberfläche ersetzt die Fachwerkzeuge nicht. Ihre Handler laden das aktive Projektmanifest sowie bei Bedarf die Laufzeitkonfiguration und delegieren anschließend unter anderem an `npm` und `Vite`, `Uvicorn` und `Pytest`, `Cargo` und `Tauri`, `Docker Compose`, `Alembic` oder `PyGitIndex`. Damit wird die in Kapitel 2 formulierte Anforderung nach gemeinsamen Entwicklungsbefehlen adressiert, während die technischen Subsysteme eigenständig bleiben. Die geführte `console` ruft dieselben öffentlichen Befehle als Kindprozesse auf und bildet daher eine alternative Bedienoberfläche, kein zweites Tooling-System.

Erwartbare Bedienfehler werden an der jeweiligen Grenze behandelt: Unbekannte Parserargumente zeigen Hilfe, Fehlermeldung und nächsten Hilfeschritt und enden mit Exit-Code 2; fachliche Fehlschläge enden regelmäßig mit Code 1. Hilfe, erfolgreiche Ausführung und reine Warnzustände liefern Code 0. Bekannte Profil- und Generierungsfehler werden ohne Traceback ausgegeben. Nur die letzte Sicherheitsgrenze für unerwartete Exceptions protokolliert zusätzlich den Stacktrace; ein dort ankommender Abbruch per Tastatur erhält Code 130. Diese Statuskonvention ermöglicht sowohl interaktive Diagnose als auch die Auswertung durch CI-Workflows (kleiveist, 2026a, 2026c).

5.2 Projektinitialisierung und Generierung

`python tools/control.py init` leitet aus dem in Kapitel 4 beschriebenen Modell ein eigenständiges Produktprojekt ab. Ohne `-profile` führt der Befehl durch Profil und kompatibel, als auswählbar markierte Capabilities. Nicht-interaktiv können Profil und wiederholbare oder kommaseparierte `-with`-Angaben gesetzt werden. Hinzu kommen Projektname, optionaler Slug, Zielverzeichnis und Dry-Run. Sobald die Identität eines Tauri-Projekts angepasst wird, verlangt der Generator außerdem einen gültigen Reverse-Domain-Identifizierer (kleiveist, 2026g, 2026l).

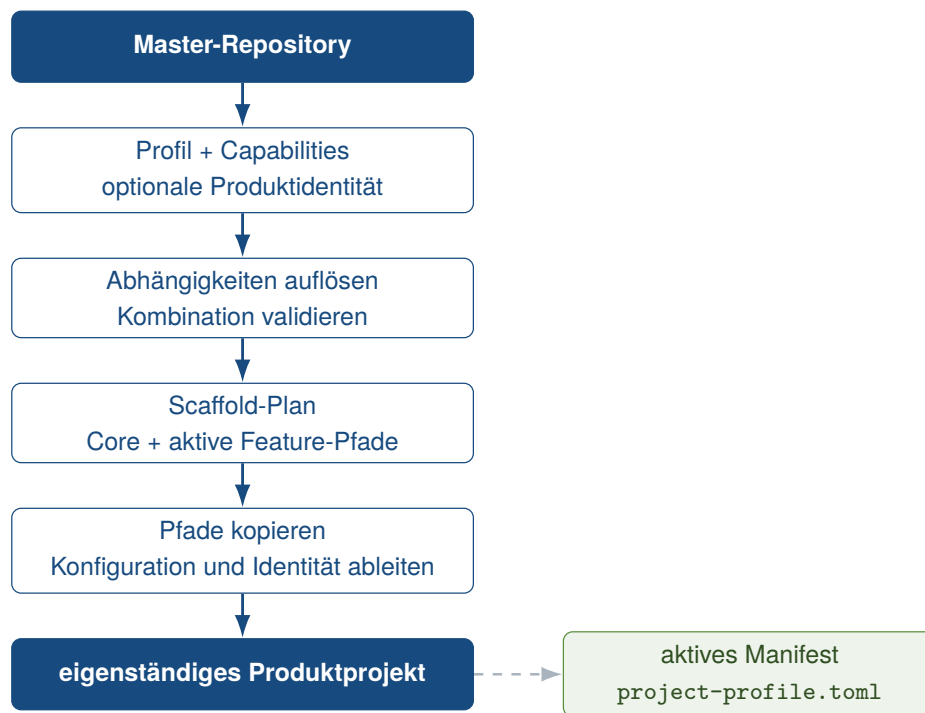


Abbildung 12: Workflow der profilgesteuerten Projektgenerierung

Quelle: Eigene Darstellung auf Grundlage von kleiveist (2026g) und kleiveist (2026l).

Technisch lädt `init` den Katalog, löst das Preset und transitive optionale Abhängigkeiten auf und bildet daraus einen geordneten Scaffold-Plan aus Core- und aktiven Feature-Pfaden. Vor einem Schreibzugriff werden Quellpfade und Ziel geprüft. Anschließend werden die ausgewählten Dateien ohne transiente Build- und Umgebungsverzeichnisse kopiert; generiert werden das aktive `project-profile.toml`, das Frontend-Profilmodul und eine featurebezogene `.env.example`. Bei angegebener Produktidentität passt die Nachbearbeitung die tatsächlich vorhandenen Frontend-, Backend-, Compose-, Tauri- und Cargo-Metadaten an. Webprofile verlieren zusätzlich Tauri-Skript und `-CLI` aus ihren kopierten npm-Metadaten.

Ein Ziel im Master ist nur unter `.generated/` zulässig; alternativ kann ein externes `-target-dir` gewählt werden. Das Repository selbst und nichtleere Ziele werden abgewiesen, während `-dry-run` nur den Plan ausgibt. Master-Template und erzeugtes Produktprojekt bleiben somit getrennt. Die Auswahl- und Kopierlogik ist auf eine deterministische Ableitung aus Katalog, Profil, Capabilities und Identität ausgelegt; eine empirische Aussage zur Reproduzierbarkeit bleibt Kapitel 6 vorbehalten. Als bereits in Abschnitt 4.7 festgestellte Implementierungsgrenze filtert nur der Dialog nach `selectable`; eine direkte `-with database`-Angabe wird für Backendprofile akzeptiert, weil die CLI dort lediglich `optional`

prüft (kleiveist, 2026k).

5.3 Systemprüfung und Installation

Der read-only ausgelegte `doctor` prüft den laufenden Python-Interpreter, Node.js, npm und npx sowie das optionale `uv`, lädt Profil und wirksame Konfiguration und kontrolliert relevante Projektpfade, Abhängigkeiten und Ports. Deaktivierte Schichten gelten nicht als Fehler. Cloudprofile ergänzen Containerprüfungen, wobei fehlendes Docker im allgemeinen Doctor nur warnt; erst `container doctor` behandelt es als Voraussetzung. Datenbankverbindung und native Desktop-Bibliotheken besitzen mit `db doctor -connect` beziehungsweise `tauri doctor` bewusst separate, vertiefende Diagnosewege. Letzterer prüft zusätzlich Rust/Cargo, Tauri-CLI und plattformspezifische Bibliotheken (kleiveist, 2026i, 2026l).

`install` verwendet für das Frontend bei vorhandenem Lockfile `npm ci`, andernfalls `npm install`. Für ein aktives Backend legt es `backend/.venv` an und installiert die Basisanforderungen sowie profilabhängig Datenbank- und PostgreSQL-Ergänzungen; `uv` ist ein optionaler schneller Pfad, für den ein `pip/venv`-Fallback existiert. Ferner wird bei Bedarf eine kleine Tooling-Umgebung für Pytest eingerichtet. Chromium wird nur installiert, wenn Playwright im Projekt konfiguriert ist. Deaktivierte Frontend- oder Backendschichten werden übersprungen, und eine lokale `.env` wird weder erzeugt noch verändert. Native Betriebssystem- und Rust-Voraussetzungen bleiben Aufgabe von `tauri install`. Damit bündelt der Standardweg die relevanten npm- und Python-Schritte, ohne jeden Sonderworkflow vorwegzunehmen.

Die README nennt Git, Python ab Version 3.11 und Node.js ab Version 20 als Grundvoraussetzungen sowie Rust Stable für Desktopprofile. Der allgemeine Doctor prüft jedoch weder Git noch diese Versionsbereiche; auch der Tauri-Doctor meldet jeden vorhandenen aktiven Rust-Toolchain als gültig. Die festgelegten Python- und Node-Hauptversionen werden dagegen in den CI-Workflows explizit eingerichtet. Dies ist eine Grenze zwischen dokumentierter Voraussetzung und lokaler Durchsetzung, nicht eine zusätzliche Tooling-Funktion (kleiveist, 2026c, 2026k).

5.4 Entwicklungsbetrieb

Der vorgesehene Entwicklungszyklus verbindet Diagnose und Installation mit den späteren Qualitäts- und Auslieferungsschritten, ohne daraus einen einzelnen monolithischen Befehl zu machen. Abbildung 13 zeigt den Standardpfad und kennzeichnet Desktop- sowie Buildvarianten als profilabhängige Abzweigungen.

`run` löst zunächst Konfiguration und Ports auf. Für alle gegenwärtigen Profile startet es den Vite-Entwicklungsserver; nur ein aktives Backend ergänzt Uvicorn. Web-only und Desktop-local führen daher keinen FastAPI-Prozess aus. Standardmäßig laufen die Dienste im Vordergrund, ihre Ausgaben werden zusammengeführt und ein Abbruch beendet die erfassten Prozessgruppen. `-detach` schreibt dagegen PID- und Logdaten nach `tools/.runtime`. `stop` beendet diese erfassten Prozesse und untersucht standardmäßig zusätzlich die konfigurierten Ports der aktiven Schichten. Dadurch kann es auch extern gestartete Listener auf genau diesen Ports beenden; `-tracked-only` begrenzt den Eingriff auf den eigenen Zustand (kleiveist, 2026a, 2026l).

Der native Desktopweg ist davon getrennt: `tauri run -foreground` delegiert an `tauri dev`, star-

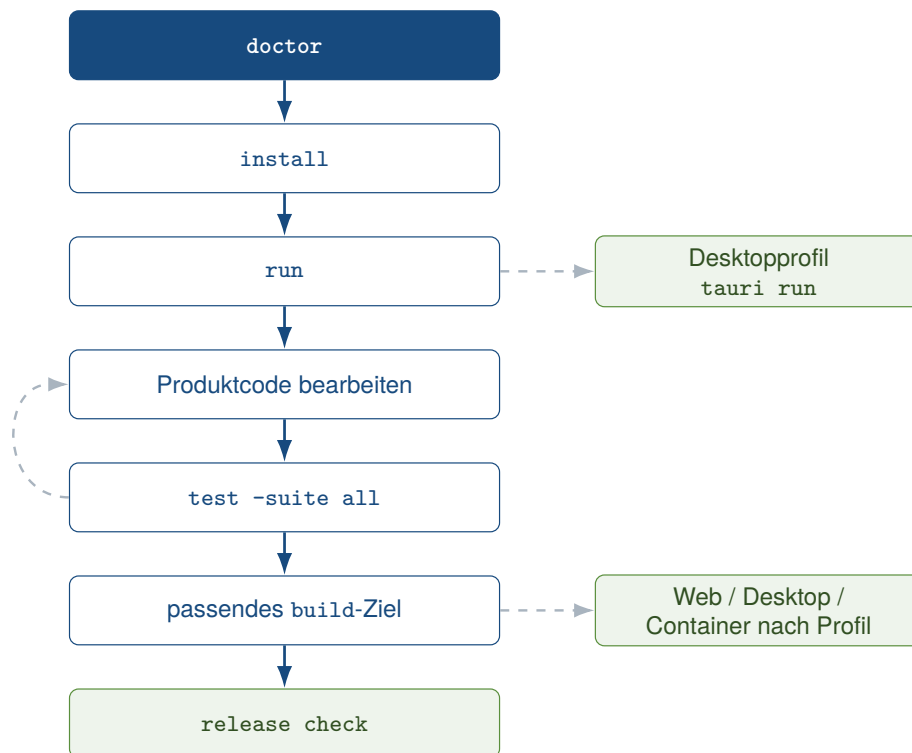


Abbildung 13: Grundstruktur des Entwicklungsworkflows

Quelle: Eigene Darstellung auf Grundlage von kleiveist (2026l) und kleiveist (2026h).

tet über dessen Konfiguration den Vite-Server und entfernt bekannte serverseitige Geheimnisse aus der Kindprozessumgebung. Ohne `-foreground` wird Tauri im Hintergrund gestartet und das Log verfolgt. Die zentrale `console` bietet für diese und die allgemeinen Entwicklungsaktionen geführte Menüs mit Bestätigungen bei schreibenden oder aufwendigen Operationen, ruft aber stets dieselben CLI-Workflows auf.

5.5 Tests und Builds

Ein leerer `test`-Aufruf zeigt zunächst die Suite-Karte; eine Ausführung erfordert `-suite`. Verfügbar sind `tools` für die Python-Toolingtests, `schema` für JSON-Schema-Beispiele, `api` und `database` für Pytest, `postgres` für Integrationstests, `frontend` für das npm-/Vitest-Skript, `e2e` für Playwright und `tauri` für Strukturprüfung, `cargo check -locked` und Rust-Tests. `test -suite all` expandiert auf diese acht Suites und ist der vollständige projektbezogene Einstieg; optionale Markdown- oder JSON-Berichte landen unter `.report/` (kleiveist, 2026a, 2026l).

Die Auswahl bleibt profilabhängig: Fehlt ein Feature im aktiven Manifest, meldet die zugehörige Suite `SKIP`; fehlen bei einem aktiven Feature Quellen oder ausführbare Voraussetzungen, entsteht `FAIL`. Die PostgreSQL-Suite wird ohne `DATABASE_URL_TEST` ebenfalls übersprungen, und die E2E-Suite nur bei vorhandener Playwright-Konfiguration ausgeführt. Für Backend-Suites prüft der Runner die virtuelle Umgebung und versucht sie bei Bedarf über `install` zu reparieren. Konfigurierte E2E-Tests können die Entwicklungsdienste selbst im Detached-Modus starten und anschließend stoppen; `-no-start` unterbindet diesen Schritt.

Die Build-Karte trennt drei Artefaktpfade. `build web` ruft das npm-Buildskript auf. Neben dem Vite-Ausgabeverzeichnis entsteht ein ZIP-Paket unter `.dist/web/`. `build desktop` ist nur bei aktivem

Tauri-Feature zulässig und reicht Ziel- und Bundlewahl an das native Tauri-Tooling weiter. Der Containerpfad ist auf Cloudprofile begrenzt und baut über Docker wahlweise Backend-, Frontend- oder beide Images. Der Befehl `container validate` prüft zuvor das Compose-Modell, ohne Dienste zu starten (kleiveist, 2026b, 2026h).

Die GitHub-Actions-Workflows verwenden dieselben öffentlichen Einstiege. Getrennte Jobs adressieren Core-Prüfungen, generierte Profilmatrizen, PostgreSQL und native Desktopziele. Lokaler Befehl und CI-Orchestrierung sind damit vergleichbar, aber nicht identisch: CI stellt Betriebssysteme, Laufzeiten und den temporären PostgreSQL-Dienst bereit. Ergebnisse dieser Ausführungen werden erst in Kapitel 6 bewertet (kleiveist, 2026c).

5.6 Release- und Packaging-Workflow

VERSION ist die zentrale Versionsquelle. `version check` vergleicht deren SemVer-Wert mit den Metadaten aller aktiven Frontend- und Tauri-Komponenten; `version sync` ist der davon getrennte, schreibende Abgleich. `release check` erzeugt und veröffentlicht selbst keine Artefakte. Das Gate kombiniert Versionsprüfung, Produktidentität, bei Tauri CSP und Capabilities, Tagbezug und sauberen Git-Arbeitsbaum. Bei Tauri kommt die Anwesenheit externer Signing-Variablen hinzu. Fehlende Signing-Konfiguration bleibt eine Warnung; inkonsistente Versionen, ein unpassender Release-Tag, ein veränderter Arbeitsbaum oder bekannte Template-Platzhalter im abgeleiteten Projekt sind Fehler. Nur das Master-Repository akzeptiert seine kanonische Template-Identität anhand des dort vorhandenen Profilmatrix-Workflows (kleiveist, 2026a, 2026h).

Der Webpfad liefert den statischen Vite-Build und das ZIP-Paket. Für Desktop wählt ein Aufruf eine Plattformstrategie; Tauri baut auf dem jeweiligen Host native Windows-, macOS- oder Linux-Bundles. Zusätzliche Strategien existieren für ein portables Windows-ZIP und einen Linux-basierten Cross-Build. Die CI-Matrix erzeugt kurzlebige, ausdrücklich unsignierte Prüfarbeitsprodukte auf den drei nativen Betriebssystemen.

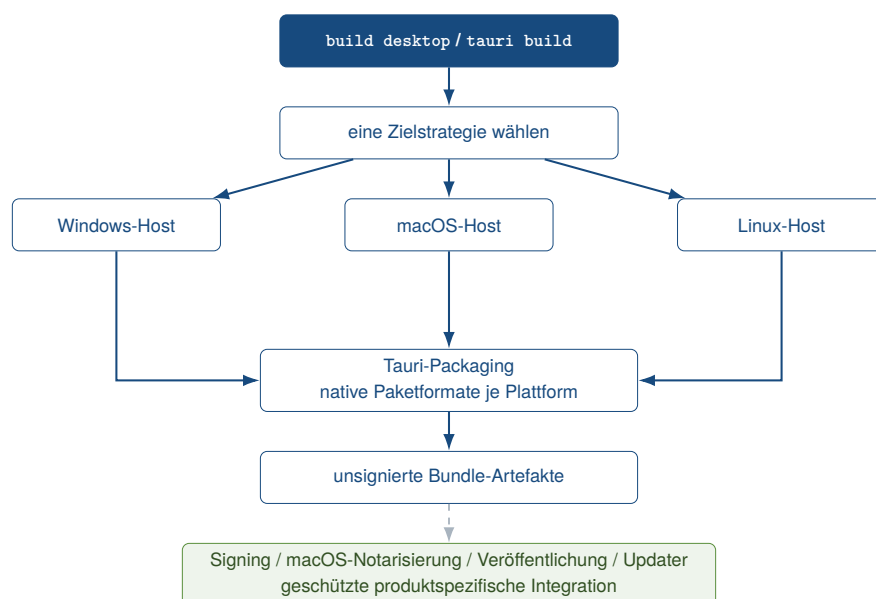


Abbildung 14: Plattformübergreifendes Modell für Desktop-Releases

Quelle: Eigene Darstellung auf Grundlage von kleiveist (2026h).

Signierung, macOS-Notarisierung, Veröffentlichung, App-Stores und die Aktivierung eines Updaters sind nicht Teil der generischen Baseline. Sie benötigen Produktidentität, geschützte Zugangsdaten und produktspezifische Freigaberegeln. Der Release-Workflow für Tags oder manuelle Auslösung führt Tests, Web- und Container-Build, Release-Gate und anschließend die unsignierte Desktopmatrix zusammen, enthält aber weder Veröffentlichungs- noch Deployment-Job (kleiveist, 2026c, 2026h).

Für Cloudprofile kontrollieren `container doctor` und `container validate` Docker, Compose und die Deployment-Dateien; `build container` baut versionierte Backend- und Frontend-Images. Die Compose-Basis dient der lokalen Produktionssimulation und hält PostgreSQL in einem optionalen Compose-Profil. Sie legt keinen Ziellanbieter fest und ersetzt weder produktive Secret-Verwaltung noch einen kontrollierten Alembic- Migrationsjob. Docker-/Deployment-Basis und fertige AWS-, Azure- oder GCP-Architektur sind daher ausdrücklich nicht gleichzusetzen (kleiveist, 2026b, 2026e).

5.7 Entwickler-Onboarding

Die README trennt zwei Einstiege. Für Produktentwicklung wird das Master-Repository geklont, mit `doctor` geprüft und mit `init` ein Profilprojekt erzeugt. Erst im Zielverzeichnis folgen erneut `doctor`, dann `install` und `run`; Produktcode, Konfiguration, Tests und Commits gehören ab diesem Punkt in das abgeleitete Repository. Template-Wartung arbeitet dagegen direkt an Baseline, Profilen oder Tooling des Masters und führt dort mindestens Diagnose, Installation und den vollständigen Testweg aus. Diese Trennung verhindert, dass Produktarbeit versehentlich die gemeinsame Vorlage verändert (kleiveist, 2026g, 2026k).

Als Grundausstattung dokumentiert das Projekt Git, Python 3.11 oder neuer und Node.js 20 oder neuer mit npm. Desktopprofile ergänzen Rust Stable. Hinzu kommen die plattformspezifischen Tauri-Voraussetzungen. Docker/Compose wird erst für Containerpfade benötigt, ein erreichbares PostgreSQL nur für die optionale PostgreSQL-Capability und PyGitIndex nur zur Regeneration der Dokumentationsnavigation. Diese Abstufung entspricht den getrennten Diagnosewegen und verhindert, optionale Werkzeuge als Blocker jedes Profils zu behandeln.

Die Navigation ist ebenfalls aufgabenbezogen: Die README beschreibt den Ersteinstieg, der Tooling Guide Befehle und Diagnose, Project Profiles das Strukturmodell und Runtime Configuration die Wertepriorität. CI-, Container- und Release-Dokumente führen zu automatisierten Prüfungen und Auslieferungsgrenzen weiter. Damit reduziert das Repository die Zahl unterschiedlicher öffentlicher Setup-Pfade; eine messbare Verkürzung des Onboardings oder Produktivitätssteigerung folgt daraus noch nicht und bleibt der Bewertung in Kapitel 7 vorbehalten (kleiveist, 2026c, 2026h, 2026i, 2026l).

6 Qualitätssicherung und Verifikation

6.1 Teststrategie

Die Verifikation folgt dem in Abschnitt 1.5 festgelegten Fallstudienprinzip: Eine Beobachtung wird einem Projektartefakt, einer ausgeführten Prüfung und einem commitgebundenen Ergebnis zugeordnet. Damit gilt weder vorhandener Testcode als Ausführungsnachweis noch eine Workflow-Datei als erfolgreicher CI-Lauf. Für die Qualitätsmerkmale aus Abschnitt 2.4 werden zuerst Ausführungsergebnisse, danach das Abnahmeprotokoll und erst anschließend Testcode, Konfiguration, Implementierung

und Dokumentation als Evidenz herangezogen (ISO/IEC, 2023; Runeson & Höst, 2009).

Die gemeinsame Baseline wird auf vier Ebenen geprüft: Die Tooling-Suite deckt CLI, Profilauflösung, Generator, Konfiguration, Release- und Repository-Hygiene sowie die Workflow-Struktur ab. Komponentenprüfungen betreffen JSON-Schema, FastAPI und SQLAlchemy sowie Vitest im Frontend und die Tauri-/Rust-Prüfkette. Integrationsprüfungen verbinden PostgreSQL 16 mit Konfigurationsdiagnose, Alembic-Migration und realer Verbindung. Schließlich erzeugen Buildprüfungen Web-Bundles, Container-Images oder native Pakete (kleiveist, 2026c).

Der Umfang wird aus dem Profilmodell in Kapitel 4 abgeleitet. Frontend- und Tooling-Prüfungen sind gemeinsam; Backend-, Tauri- und PostgreSQL-Suiten laufen nur bei aktivierter Capability. Ein deaktivierter Gegenstand ist daher *nicht anwendbar* und kein Fehler. *PASS* bezeichnet eine erfolgreich ausgeführte Prüfung, *FAIL* ein ausgeführtes und nicht erfülltes Kriterium und *nicht verifiziert* das Fehlen hinreichender Evidenz. So werden A-01 und A-03 aus Tabelle 1 durch Generierung und Strukturprüfung, A-04 durch die öffentliche Tooling-Schnittstelle und A-07 durch gültige sowie absichtlich abgewiesene Capability-Kombinationen operationalisiert. Abbildung 15 ordnet diese Prüfungen in die Nachweiskette ein.

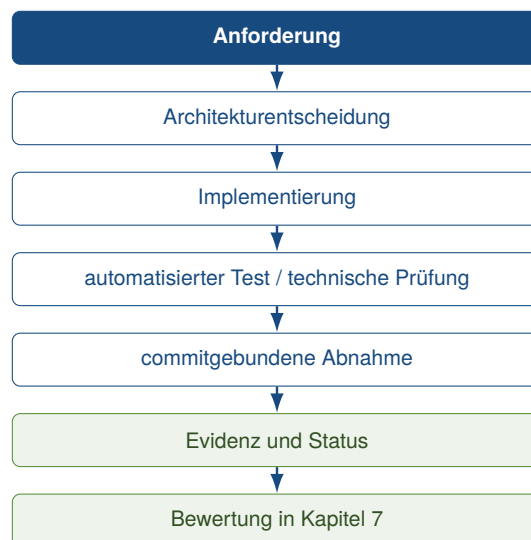


Abbildung 15: Nachweiskette der technischen Verifikation

Quelle: Eigene Darstellung.

6.2 Continuous Integration

Vier GitHub-Actions-Workflows reagieren auf Pull Requests, Pushes nach `main` und manuelle Auslösung: *Core CI* trennt Tooling, Backend/Schema, Frontend/Web-Build und Container-Build in Ubuntu-Jobs; *Profile Matrix* generiert und prüft die fünf Profile unter Ubuntu; *PostgreSQL Integration* nutzt einen isolierten PostgreSQL-16-Service für Master und drei kompatible Kombinationen; *Desktop CI* baut unsignierte Artefakte nativ unter Linux, macOS und Windows. Die separate *Release Validation* läuft nur manuell oder für Tags der Form `v*.*.*`. Alle Definitionen besitzen nur Leserechte und rufen die auch lokal nutzbare Schnittstelle `tools/control.py` auf (kleiveist, 2026c).

Die am 9. August 2026 abgefragten externen Läufe vom 7. August beziehen sich eindeutig auf den untersuchten HEAD `a4487ac`. Ihr Ergebnis ist nicht insgesamt grün: In *Core CI* bestanden Backend

und Schema, Frontend samt Web-Build sowie Compose-Validierung und beide Image-Builds. Der Job für Tooling, Profile und Konfiguration scheiterte. In der *Profile Matrix* bestanden `web-only` und `web-cloud`; alle drei Desktopprofile scheiterten im Schritt *Test generated project*. Die Basisintegration mit PostgreSQL und `web-cloud + postgres` bestanden. Die beiden Desktopkombinationen mit PostgreSQL scheiterten dagegen beim Projekttest. In *Desktop CI* bestanden die Paketierungen unter Linux und macOS; der Windows-Job scheiterte bereits bei der Frontend-Installation (GitHub, 2026a, 2026c, 2026d, 2026e).

Ein lokaler Kontrolllauf am 9. August 2026 führte auf demselben HEAD unter Python 3.13.14 alle 194 Tooling-Tests erfolgreich aus; auch Schema, API und SQLAlchemy-Einheitstests bestanden lokal. Dies lokalisiert den CI-Widerspruch, ersetzt aber weder den fehlgeschlagenen externen Job noch die mangels Node.js, Rust und Docker auf diesem Host nicht ausgeführten Prüfpfade. Für *Release Validation* existiert auf diesem Commit kein Run. Struktur, Ausführung und Ergebnis bleiben somit getrennte Aussagen.

6.3 Abnahme und technische Verifikation

Das finale Abnahmeprotokoll bewertet den funktional geprüften Commit `1cdfb6e` (vollständige SHA in der zitierten Abnahme) als *PASS WITH OPEN EXTERNAL ITEMS* und *READY FOR CASE STUDY*. Dies bezeichnet die Annahme der Repository-Baseline für die Fallstudie, nicht Produktionsreife oder eine Freigabe externer Deployments. Der dokumentierte Tooling-Lauf umfasste dort 189 bestandene Tests; die Zahl beschreibt allein diese Suite auf diesem Commit (kleiveist, 2026j).

Die lokale Abnahme generierte alle fünf Basisprofile, installierte ihre Abhängigkeiten, führte profilabhängige Diagnosen und Tests sowie jeweils den Web-Build aus. Für die drei Desktopprofile entstand zusätzlich ein Linux-DEB. Die Doctor-Hinweise der Cloudprofile betrafen das optional fehlende lokale Compose-Werkzeug; mit bereitgestelltem Compose bestand die strikte Validierung. Separat bestanden `web-cloud`, `desktop-cloud` und `full-platform` mit PostgreSQL einschliesslich Verbindungstest, Alembic `upgrade/current`, Backend-, Frontend- und Datenbanktests sowie Web-Build; die Desktopvarianten wurden ebenfalls als Linux-DEB paketierte. `web-only + postgres` und `desktop-local + postgres` wurden erwartungsgemäss mit Exitcode 1 vor der Zielerzeugung abgewiesen. Dieses Fehlerverhalten gilt deshalb als PASS (kleiveist, 2026j).

Abbildung 16 zeigt die beiden Evidenzstände, Tabelle 5 löst die lokale Abnahme nach Profil und Prüfdimension auf und stellt ihr die aktuelle externe CI gegenüber. Die neueren CI-Fehler entwerten die commitgebundene Abnahme nicht, verhindern aber ihre Übertragung auf `a4487ac` als vollständig grünen Remote-Nachweis.

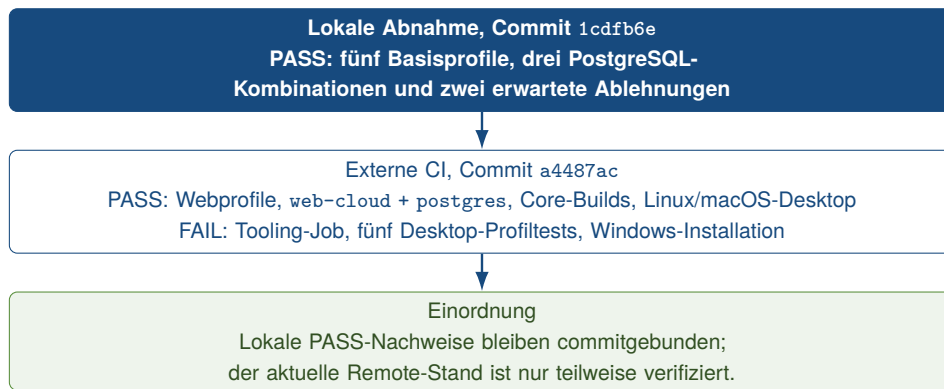


Abbildung 16: Profilverifikation nach Evidenzstand und Commit

Quelle: Eigene Darstellung auf Grundlage von kleiveist (2026j) und den aktuellen GitHub-Actions-Läufen.

Tabelle 5: Profilbezogene technische Verifikation

Profil / Kombination	Gen.	Install / Doctor	Tests	Web	Desktop	PG / Migr.	CI auf a4487ac
web-only	P	P	P	P	–	–	P
web-cloud	P	H	P	P	–	–	P
desktop-local	P	P	P	P	P (Linux DEB)	–	F: Profiltest
desktop-cloud	P	H	P	P	P (Linux DEB)	–	F: Profiltest
full-platform	P	H	P	P	P (Linux DEB)	–	F: Profiltest
web-cloud + postgres	P	P	P	P	–	P	P
desktop-cloud + postgres	P	P	P	P	P (Linux DEB)	P	F: Profiltest
full-platform + postgres	P	P	P	P	P (Linux DEB)	P	F: Profiltest

Legende: P = PASS; H = PASS WITH NOTE; F = ausgeführte Prüfung fehlgeschlagen; – = nicht anwendbar. Die sechs mittleren Ergebnisspalten stammen aus der lokalen Abnahme auf 1cdfb6e; die letzte Spalte gibt den profilbezogenen GitHub-Actions-Run auf a4487ac wieder. web-only + postgres und desktop-local + postgres wurden in der lokalen Abnahme vor der Zielerzeugung korrekt abgewiesen (P). Quelle: kleiveist (2026j) sowie GitHub (2026d, 2026e).

6.4 Reproduzierbarkeit

Zwei unabhängige Generierungen von desktop-cloud + postgres mit identischen Eingaben waren im Abnahmelau auf 1cdfb6e bei rekursivem Vergleich bytegleich. Zusätzlich schrieb `init -dry-run` kein Ziel; nichtleere Ziele wurden abgelehnt und inkompatible Capabilities vor dem ersten Schreibvorgang validiert. Der Nachweis betrifft genau diese Kombination und diese Umgebung, nicht den gesamten Zustandsraum (kleiveist, 2026j; Lamb & Zacchiroli, 2022).

Für Abhängigkeiten liegen `frontend/package-lock.json` und `src-tauri/Cargo.lock` vor; Installation beziehungsweise Cargo-Prüfungen verwenden `npm ci` und `-locked`. Drei produktive Python-Lockfiles fixieren den vollständigen Abhängigkeitsgraphen samt Hashes für die Container-Baseline Python 3.11. Deren hashgebundene Installation wurde im Abnahmeprotokoll geprüft (kleiveist, 2026b, 2026c). Die Reproduktion bleibt dennoch von Betriebssystem, Toolchain, Registries, Netzwerk und Plattform-SDKs abhängig; native Ergebnisse sind daher plattformspezifisch und keine absolute Buildgarantie.

6.5 Security und Konfigurationsgrenzen

Die in Abschnitt 3.6 beschriebene Priorität CLI-Override vor Prozessumgebung, `.env` und Default ist durch einzelne Tooling-Tests belegt. Dieselbe Suite prüft ungültige Umgebungen, Ports und CORS-Ursprünge, die profilabhängige Erzeugung von `.env.example` und die Pflicht zu `DATABASE_URL` nur bei PostgreSQL. Der aktuelle lokale Lauf dieser Tests bestand; der externe Tooling-Job auf demselben Commit bleibt wie in Abschnitt 6.2 beschrieben fehlgeschlagen (GitHub, 2026a; kleiveist, 2026i).

Die geprüfte Grenze lässt nur die öffentliche `VITE_API_BASE_URL` in das Frontend gelangen. Tests werfen öffentlich deklarierte Secrets, entfernen `DATABASE_URL` aus Frontend- und Tauri-Umgebungen und maskieren Datenbankpasswörter einschliesslich kodierter Varianten in Diagnosen. Profile ohne PostgreSQL erzeugen keine Datenbankvariable. Backend- und aktuelle CI-Tests decken auSSerdem `/api/health` und `/api/ready`, Fehlerantworten sowie datenbankunabhängige Liveness ab; der Release-Check prüft Tauri-CSP und Capability-Metadaten (kleiveist, 2026h, 2026j).

Diese Befunde verifizieren nur Konfigurations-, Secret- und Desktop-Grenzen der Baseline. Authentifizierung, Rollen- und Berechtigungskonzepte, produktive Credentials, TLS-Terminierung und providerspezifische Härtung sind gemäss Abschnitt 2.5 produktspezifisch. Eine vollständige Produktsicherheitsbewertung ist daher nicht Gegenstand dieser Verifikation.

6.6 Extern verbleibende Prüfungen

Die offenen Punkte in Tabelle 6 sind keine einheitliche Fehlerklasse. *FAIL* bedeutet, dass eine ausgeführte Prüfung ihr Kriterium nicht erfüllte; *nicht verifiziert* bezeichnet fehlende hinreichende Evidenz; *auSSerhalb des Scopes* kennzeichnet eine bewusste Produktgrenze. So sind die aktuellen roten CI-Jobs reale Fehlerbefunde, während ein nie gestarteter Compose-Verbund nicht als fehlgeschlagen gelten kann.

Der aktuelle Core-Containerjob belegt zwar Compose-Validierung und den Bau beider Master-Images, aber weder `docker compose up` noch eine produktive Bereitstellung. Linux- und macOS-Pakete sind auf `a4487ac` unsigniert gebaut; Windows ist wegen des fehlgeschlagenen Installationsschritts nicht paketierte. Branch Protection konnte über die öffentliche API ohne Berechtigung nicht gelesen werden. Signing, Notarisierung, Updater, produktive Zugangsdaten sowie Authentifizierung/RBAC benötigen schliesslich ein konkretes Produkt und geschützte externe Infrastruktur (GitHub, 2026a, 2026c; kleiveist, 2026h).

Tabelle 6: Offene und externe Verifikationspunkte auf a4487ac

Prüfpunkt	Status	Grund	erforderlicher Nachweis
Tooling- und Desktop-Profil-CI	FAIL	aktuelle Jobs enden in ausgeführten Testschritten mit Exitcode 1	erfolgreiche Wiederholung auf dem korrigierten Commit
Windows-Paket	FAIL	Frontend-Installation im nativen Windows-Job fehlgeschlagen	erfolgreicher nativer Build und Artefakt-Upload
Release Validation	nicht verifiziert	kein manueller oder Tag-Run vorhanden	erfolgreicher, nicht publizierender Run auf festem Commit
Compose-Start	nicht verifiziert	nur Modell und Master-Images in CI geprüft	<code>docker compose up</code> , Health-/Ready-Nachweis und kontrollierter Abbau
Branch Protection	nicht verifiziert	öffentliche API verlangt Authentifizierung	autorisierter Nachweis der Required Checks für <code>main</code>
Signing, Notarisierung, Updater	außerhalb des Scopes	produktspezifische Identität und geschützte Credentials fehlen bewusst	geschützte native Release-Pipeline je Zielplattform
Produktivbetrieb und Auth/RBAC	außerhalb des Scopes	Cloud, Geschäftsmodell und Sicherheitsmodell sind produktspezifisch	produktbezogene Abnahme einschließlich Credentials und Deployment

Quelle: Eigene Zusammenstellung aus kleiveist (2026h, 2026j) und den aktuellen GitHub-Actions-Läufen.

7 Bewertung des Technologie-Templates

7.1 Produktivitätswirkung

Die Bewertung folgt einer Soll–Ist–Evidenz–Logik. Aus den Anforderungen in Kapitel 2 werden Wiederverwendbarkeit, Standardisierung, kontrollierte Variabilität, Wartbarkeit, Testbarkeit, Reproduzierbarkeit, Entwicklerfreundlichkeit, Erweiterbarkeit und Plattformabdeckung als zentrale Kriterien übernommen. Komplexität, Betriebsreife und Dokumentierbarkeit dienen ergänzend dazu, Nutzen und Folgekosten einzuordnen. Als Ist-Zustand gelten die in den Kapiteln 3 bis 5 beschriebenen Mechanismen. Technische Evidenz liefern die in Kapitel 6 angelegte Verifikation und das projektinterne Abnahmeprotokoll; dessen Befunde sind dokumentierte Projektnachweise und keine unabhängige Replikation (kleiveist, 2026j).

Produktivität wird hier nicht als Menge erzeugten Codes verstanden. Sie umfasst vielmehr den Aufwand bis zu einem lauffähigen Ausgangsprojekt, die Zahl wiederkehrender Einrichtungsentscheidungen, einen einheitlichen Entwicklungsfluss sowie den späteren Pflegeaufwand. Diese mehrdimensionale Betrachtung entspricht der Forschung, nach der Entwicklerproduktivität nicht durch eine einzelne Aktivitätskennzahl angemessen beschrieben werden kann (Forsgren et al., 2021). Konzeptionell lässt sich die Wirkung daher als

$$\text{Produktivitätsnutzen} = \text{vermiedener Projektaufwand} - \text{Template-Pflegeaufwand} - \text{zusätzliche Komplexität}$$

auffassen. Die Beziehung strukturiert die Diskussion, ist aber mangels Zeit- und Aufwandsmessungen nicht numerisch auswertbar.

Beim *Projektstart* sind Profilwahl, Scaffold-Plan, Generierung, Systemprüfung und Installation über `control.py` gebündelt. Das Abnahmeprotokoll dokumentiert Generierung, Installation und Diagnose für alle fünf Profile sowie die sichere Ablehnung inkompatibler PostgreSQL-Auswahlen (kleiveist,

2026j)). Dies ist direkte technische Evidenz für automatisierte und vereinheitlichte Setup-Schritte auf dem dort geprüften Commit. Die gemischten CI-Ergebnisse des später untersuchten HEAD begrenzen die Übertragbarkeit (Abschnitt 6.2). Aus den Mechanismen folgt dennoch ein plausibles Potenzial für weniger manuelle Einrichtung und Fehlentscheidungen; eine konkrete Zeitersparnis ist damit nicht bewiesen.

In der *laufenden Entwicklung* reduzieren profilabhängige Einstiege für Start, Test und Build die Zahl projektspezifischer Bedienpfade. Gemeinsames Frontend, Konfigurationsvertrag und dokumentierte Komponentenverantwortung fördern zudem die Wiederverwendung. Dem stehen die weiterhin erforderlichen Fachwerkzeuge und plattformspezifischen Voraussetzungen gegenüber. In der *Wartung* kann eine Änderung an der zentralen Baseline zukünftigen Projektstarts zugutekommen; Profile, Generator, Testmatrix und Dokumentation müssen dafür jedoch gemeinsam gepflegt werden. Bereits generierte Produktprojekte erhalten diese Änderungen nicht automatisch.

Quantitative Produktivitätsevidenz liegt somit nicht vor. Eine spätere kontrollierte Evaluation sollte vergleichbare Vorhaben mit und ohne Template beobachten und mindestens die Zeit bis zum ersten lauffähigen Build und zur ersten Fachfunktion, Setup-Fehler, Onboarding-Zeit, fehlgeschlagene Builds, den Anteil wiederverwendeter Basiskomponenten und die Pflegezeit der Baseline erfassen. Erst solche Messungen erlauben eine belastbare Aussage über die Grösse des Produktivitätseffekts.

7.2 Stärken

Die architektonische Hauptstärke ist die Verbindung einer gemeinsamen Baseline mit expliziten Verantwortungsgrenzen. Frontend, Backend, Desktop-Schicht, Persistenz und Tooling bleiben getrennte Bausteine; zugleich wird das Frontend profilübergreifend verwendet. Damit adressiert die Architektur die Anforderungen A-02 und A-05, ohne jede Variante mit allen Laufzeitkomponenten zu belasten (Kapitel 3).

Bei der Variabilität bilden fünf Profile wiederkehrende Plattformzuschnitte ab. Abhängigkeiten verhindern insbesondere PostgreSQL ohne Backend, während die Capability aus kompatiblen Profilen herausgelöst bleibt. Die dokumentierte Abnahme der fünf Basisprofile, der drei zulässigen PostgreSQL-Kombinationen und der beiden unzulässigen Kombinationen stützt diese Stärke für den aktuellen, begrenzten Feature-Katalog (kleiveist, 2026j). Sie belegt nicht, dass beliebig viele künftige Capabilities ebenso beherrschbar wären.

Prozessual schafft `control.py` einen gemeinsamen Einstieg, ohne npm, Pytest, Cargo, Tauri, Docker oder Alembic zu verbergen. Profilabhängiges Überspringen nicht vorhandener Schichten verbindet eine einheitliche Befehlsoberfläche mit den tatsächlichen Projektgrenzen (Kapitel 5). Für Qualität und Nachvollziehbarkeit sprechen das versionierte Profilmanifest, der Konfigurationsvertrag, Tests und Buildpfade sowie die dokumentierte byte-identische Generierung zweier `desktop-cloud + postgres`-Projekte bei gleichen Eingaben (kleiveist, 2026j). Der Nachweis zur Reproduzierbarkeit ist damit konkret, aber auf eine untersuchte Kombination und Umgebung begrenzt.

Tabelle 7 verdichtet diese Stärken gemeinsam mit ihren jeweiligen Geltungsgrenzen. Sie trennt beobachtete Mechanismen und dokumentierte Prüfergebnisse von der daraus abgeleiteten Bedeutung.

Tabelle 7: Evidenzbasierte Gegenüberstellung von Stärken und Grenzen

Bereich	Stärke und Evidenz	Grenze und Evidenz	Bedeutung
Architektur	Getrennte Frontend-, Backend-, Tauri- und Persistenzgrenzen; gemeinsames Frontend.	Shared Schemas werden nicht automatisch von Frontend und Backend konsumiert.	Wiederverwendung bei klarer Zuständigkeit; Vertragsänderungen bleiben manuell.
Variabilität	Fünf deklarative Profile; Abhängigkeiten und drei zulässige PostgreSQL-Kombinationen sind abgenommen.	Nur eine auswählbare Capability; direkte CLI trennt <code>selectable</code> nicht strikt.	Kontrollierte Auswahl im aktuellen Katalog, begrenzte Aussage für Erweiterungen.
Tooling	Gemeinsamer profilabhängiger Einstieg für Diagnose, Installation, Start, Test und Build.	Mehrere Fachwerkzeuge und native Voraussetzungen bleiben erforderlich.	Weniger unterschiedliche Bedienpfade, aber kein technologieunabhängiger Workflow.
Qualität	Tests, Profilmatrix, Buildpfade und Release-Gate sind im Abnahmeprotokoll erfasst.	Aktuelle Tooling-, Desktopprofil- und Windows-Jobs schlagen fehl; Branch Protection ist nicht nachgewiesen.	Gute lokale Prüfbasis, aber kein durchgehend grüner Remote-Nachweis.
Reproduzierbarkeit	Zwei gleich konfigurierte Projekte waren im dokumentierten Versuch byte-identisch.	Nachweis nur für eine Kombination und eine Untersuchungsumgebung.	Konkrete Evidenz ohne Anspruch auf vollständige plattformübergreifende Reproduzierbarkeit.
Betrieb und Plattformen	Web- und Image-Builds, PostgreSQL-Integration sowie unsignierte Linux-/macOS-Pakete sind dokumentiert.	Compose-Start, produktiver Cloudbetrieb, Windows-Paket, Signing und Notarisierung bleiben offen.	Template-Reife ist nicht mit Produktionsreife gleichzusetzen.
Wartung	Core, Profile, Tooling, Tests und Dokumentation liegen in einer Baseline.	Änderungen können mehrere Profile betreffen; Produktprojekte erhalten keine automatischen Updates.	Zentrale Pflege wird gebündelt, erfordert aber eine breite Regression und Transferprozesse.

Quelle: Eigene Bewertung auf Grundlage der Kapitel 3–6 sowie des Abnahmeprotokolls und der aktuellen CI-Läufe (GitHub, 2026a, 2026c, 2026d, 2026e; kleiveist, 2026j).

7.3 Schwächen und Grenzen

Für die Einordnung sind drei Fälle zu unterscheiden. Eine *technische Schwäche* ist ein Nachteil der vorhandenen Lösung, eine *Scope-Grenze* eine bewusste Auslassung und ein *nicht verifizierter Bereich* eine fehlende Nachweisabdeckung. Erstere wirkt unmittelbar auf die Bewertung; letztere beiden sind nicht automatisch Implementierungsfehler.

Technisch erhöht die Bündelung im Master-Repository dessen interne Komplexität. Änderungen an gemeinsamen Pfaden, Generator und Feature-Katalog können mehrere Profile betreffen und verlangen Regressionstests über die Varianten. Der derzeit kleine Katalog begrenzt dieses Risiko, beseitigt es aber nicht. Hinzu kommen zwei konkrete Architekturgrenzen: Die gemeinsamen JSON-Schemata werden von Frontend und Backend noch nicht zur Laufzeit konsumiert, sodass Schnittstellenänderungen manuell synchronisiert werden müssen (Abschnitt 3.5). Zudem trennt die direkte CLI-Auswahl `optional` und `selectable` weniger streng als der interaktive Dialog (Abschnitt 4.7). Beide Punkte erzeugen Wartungs- beziehungsweise Fehlbedienungsrisiken, ohne die geprüften Presets grundsätzlich infrage zu stellen. Hinzu kommt ein Prozessbefund: Auf dem untersuchten HEAD bestanden die Webprofile in der externen Matrix, während der Tooling-Job, die Desktop-Profiltests und der Windows-Installationsschritt fehlschlagen; einem lokal bestandenen Tooling-Kontrolllauf steht damit ein widersprüchlicher Remote-Nachweis gegenüber (Abschnitt 6.2).

Die Profilgrenzen sind dagegen beabsichtigt: `web-only` enthält kein Backend, `desktop-local` keine Cloud-API, und PostgreSQL ist nur mit Backendprofilen vereinbar. Ebenso gehören Fachlogik, generische Authentifizierung oder Rollenmodelle, eine verpflichtende Datenbank, ein bestimmter Cloud-Provider sowie produktspezifische native Kommandos nicht zur Baseline. Daraus entsteht Anpas-

sungsaufwand für konkrete Produkte, aber keine Abweichung von der Abgrenzung in Abschnitt 2.5.

Nicht abschließend verifiziert sind die in Abschnitt 6.6 vorgesehenen externen Punkte. Die aktuellen CI-Läufe belegen Compose-Validierung und beide Master-Image-Builds, nicht jedoch den Start des Compose-Verbunds oder eine produktive Bereitstellung. Offen bleiben ferner Release Validation und Branch Protection. Linux- und macOS-Pakete wurden auf dem untersuchten HEAD unsigned gebaut; der Windows-Job scheiterte. Signing, macOS-Notarisierung, Veröffentlichung und produktive Cloud-Bereitstellung bleiben bewusst produkt- oder infrastrukturspezifisch (GitHub, 2026a, 2026c; kleiveist, 2026j). Daraus folgen vor allem Betriebs-, Plattform- und Prozessrisiken: Die vorliegenden Paketergebnisse lassen sich nicht auf signierte Releases und nicht auf einen produktiven Cloudbetrieb übertragen. Die Fallstudie bewertet daher eine technische Template-Baseline, nicht die Produktionsreife eines daraus entstehenden Produkts.

7.4 Wartungsaufwand

Die Single-Repository-Strategie aus Abschnitt 4.8 bündelt Core, Feature-Implementierungen, Profile, Generator, Tests und Dokumentation in einer Source of Truth. Gegenüber mehreren unabhängig gepflegten Vorlagen vermeidet dies parallele Änderungen gemeinsamer Bestandteile. Der Vorteil gilt unmittelbar für die Baseline und für künftig erzeugte Projekte; die Übertragung einer Korrektur in bereits abgeleitete Produktrepositories bleibt ein eigener, nicht automatisierter Prozess.

Diese Bündelung verlagert Aufwand in die zentrale Pflege. Node.js/TypeScript, Python, Rust, PostgreSQL, Docker und GitHub Actions verwenden unterschiedliche Dependency-Manager und Toolchains; auch Plattformvoraussetzungen und Releasezyklen unterscheiden sich. Die Vielfalt ist sachlich durch Web-, Backend-, Desktop- und Deploymentziele begründet, erhöht aber die für Aktualisierungen erforderliche Kompetenz und Testbreite. Insbesondere native Builds und externe Dienste machen die Wartung umgebungsabhängiger als bei einem reinen Web-Starter.

Auch das Variabilitätsmodell verursacht Folgekosten. Aktuell begrenzen fünf Profile, sechs Features und eine benutzerseitig auswählbare Capability den Kombinationsraum; die dokumentierte Abnahme deckt die fünf Basispfade und drei zulässige PostgreSQL-Erweiterungen ab (kleiveist, 2026j). Weitere Capabilities können zusätzliche Abhängigkeiten und zulässige Kombinationen erzeugen. Dann müssen Katalog, Generatorlogik, Konfigurationsvertrag, Testmatrix und Dokumentation gemeinsam erweitert werden. Es ist daher keine mathematische Explosion nachgewiesen, wohl aber ein mit der Variabilität wachsender Pflegebedarf.

Insgesamt ist der Wartungsaufwand als bewusste Zentralinvestition zu bewerten: Er ist höher als bei einem kleinen, technologiehomogenen Starter, kann aber wiederholte Basispflege über mehrere zukünftige Projektarten hinweg bündeln. Ob diese Investition wirtschaftlich ist, hängt von Nutzungshäufigkeit, Ähnlichkeit der Projekte und der tatsächlich gemessenen Pflegezeit ab; solche Kostendaten liegen nicht vor.

7.5 Vergleich mit alternativen Template-Strategien

Der Vergleich betrachtet Architekturstrategien, keine empirisch vermessenen Produkte. Als gemeinsame Kriterien dienen fortlaufende Wiederverwendung, Konsistenz, kontrollierte Variabilität, Einstiegskomplexität, Pflegebündelung, Testbarkeit und die Übernahme späterer Baseline-Änderungen. Diese

Kriterien folgen dem Grundgedanken systematischer Wiederverwendung und der Verwaltung gemeinsamer sowie variabler Bestandteile (Clements & Northrop, 2001; Krueger, 1992). Die Einstufungen *hoch*, *mittel* und *niedrig* bezeichnen nur die qualitative Ausprägung eines Merkmals. Bei der Einstiegs-komplexität ist *hoch* folglich ein Nachteil; die Tabelle ist kein Gesamtranking.

Separate Template-Repositories isolieren beispielsweise Web-, Desktop- und Cloudvorlagen. Eine einzelne Vorlage bleibt übersichtlich und separat testbar, gemeinsame Komponenten können jedoch divergieren und müssen in mehreren Quellen aktualisiert werden. Ein *Copy-Paste-Starter* minimiert die Vorlagenlogik und lässt das Zielprojekt nach der Kopie frei weiterlaufen. Repository-Vorlagen von GitHub illustrieren diese Trennung: Ein erzeugtes Repository beginnt mit einer eigenen, nicht verwandten Historie (GitHub, 2026b). Dies erleichtert den Start, stellt aber keinen automatischen Rückkanal für spätere Verbesserungen bereit.

Framework-spezifische Starter fokussieren einen engeren technischen Ausschnitt. *create-vite* bietet beispielsweise einfache Vorlagen für mehrere Frontendframeworks; generierte Projekte enthalten die üblichen Entwicklungs- und Buildskripte (VoidZero Inc. and Vite Contributors, 2026). Für ein kleines homogenes Frontendprojekt ist diese geringere Einstiegskomplexität vorteilhaft. Backend, Desktop-Packaging, Persistenz und ein Workflow für den Gesamtstack müssen bei Bedarf jedoch separat integriert werden.

Ein *monolithisches Universal-Template* enthält alle Bausteine in jedem Projekt. Seine Auswahl- und Generatorlogik kann einfacher sein, dafür tragen kleine Projekte ungenutzte Komponenten und eine gröSSere Ausgangsstruktur. Der untersuchte Ansatz setzt dem ein *Single-Repository mit Profilen und optionalen Capabilities* entgegen: Gemeinsame Implementierungen werden zentral gepflegt, während der Generator nur ausgewählte Pfade übernimmt. Dies bietet im untersuchten Variantenraum hohe Konsistenz und kontrollierte Flexibilität, bezahlt sie aber mit Master-, Resolver- und Matrixkomplexität. Wie beim Copy-Starter sind erzeugte Produktprojekte anschlieSSend eigenständig; zentrale Updates wirken daher nicht automatisch rückwärts.

Tabelle 8: Qualitativer Vergleich von Template-Strategien

Strategie	fortlaufende Wiederverwendung	Konsistenz	Variabilität	Einstiegs-komplexität	Pflege-bündelung	Updates bestehender Projekte
Separate Template-Repositories	mittel	mittel	mittel	niedrig	niedrig	je Vorlage beziehungsweise Projekt manuell
Copy-Paste-Starter	niedrig nach Kopie	niedrig	hoch	niedrig	niedrig	keine automatische Übernahme
Framework-spezifischer Starter	hoch im engen Scope	hoch im engen Scope	niedrig bis mittel	niedrig	hoch im Starter	meist manuell nach Generierung
Monolithisches Universal-Template	mittel	hoch	niedrig	hoch	hoch	abhängig vom gewählten Ableitungsweg
Single-Repository mit Profilen	hoch im definierten Scope	hoch	hoch, kontrolliert	mittel	hoch	im Master zentral, in abgeleiteten Projekten manuell

Quelle: Eigene qualitative Gegenüberstellung auf Grundlage von Krueger (1992), Clements und Northrop (2001), GitHub (2026b) und VoidZero Inc. and Vite Contributors (2026).

Der profilbasierte Ansatz ist somit insbesondere für wiederkehrende, technisch verwandte Vorhaben für Web, Cloud und Desktop plausibel. Für ein einmaliges, kleines Frontend kann ein Framework-Starter angemessener sein; bei bewusst vollständig unabhängigen Produktlinien können separate Repositories die organisatorische Isolation höher gewichten. Der Vergleich zeigt einen Trade-off zwi-

schen einfacher Einzelvorlage und zentral beherrschter Variabilität, keinen universellen Sieger.

7.6 Gesamtbewertung

Für die Zusammenführung werden drei qualitative Stufen verwendet. *Weitgehend adressiert* bedeutet, dass ein Anforderungskriterium durch implementierte Mechanismen und einschlägige Abnahmen im vorgesehenen Umfang gestützt wird. *Teilweise adressiert* kennzeichnet eine tragfähige Grundlage mit wesentlichen Reichweiten- oder Nachweislücken. *Nicht abschließend nachgewiesen* bedeutet, dass die vorhandene Evidenz keine belastbare Gesamtfeststellung zulässt. Die Stufen messen weder Qualität noch Produktivität numerisch.

Tabelle 9: Zusammengeführte Bewertung des Technologie-Templates

Kriterium	Evidenzbasis	Bewertung	wesentliche Einschränkung
Wiederverwendung und Standardisierung	Gemeinsamer Kern, fünf Profile, Generator und einheitliche Befehlsoberfläche	weitgehend adressiert	Gilt für den definierten Stack; keine automatische Aktualisierung abgeleiteter Projekte.
Variabilität und Erweiterbarkeit	Profil-Presets, Capability-Abhängigkeiten und geprüfte PostgreSQL-Kombinationen	weitgehend adressiert	Kleiner aktueller Katalog; Erweiterungen vergrößern Pflege- und Testumfang.
Testbarkeit und Reproduzierbarkeit	Dokumentierte Profil-, Integrations- und Buildabnahme sowie ein byte-identischer Generierungsvergleich	teilweise adressiert	Aktuelle Remote-Jobs teilweise fehlgeschlagen; Reproduzierbarkeitsversuch nur für eine Kombination.
Entwicklerfreundlichkeit und Dokumentierbarkeit	<code>control.py</code> , Diagnosewege, versionierte Manifeste und gegliederte Dokumentation	teilweise adressiert	Onboarding und Bedienaufwand nicht empirisch gemessen; mehrere Toolchains bleiben sichtbar.
Wartbarkeit	Eine zentrale Baseline mit gemeinsamen Tests und Dokumentationsregeln	teilweise adressiert	Technologievelfalt, Variantenregression und manuelle Schnittstellensynchronisierung.
Plattformabdeckung und Betriebsreife	Web-, Backend-, PostgreSQL-, Container-Image-, Linux- und macOS-Nachweise; providerneutrale Grenze	nicht abschließend nachgewiesen	Windows-Paket, Compose-Start, Signing, Notarisierung und produktives Deployment nicht belegt.

Quelle: Eigene Bewertung auf Grundlage der vorangehenden Kapitel, kleiveist (2026j) und der in Abschnitt 6.2 ausgewerteten CI-Läufe.

Bezogen auf die Fragestellungen in Abschnitt 1.4 zeigt die Fallstudie eine belastbare technische Baseline für die definierten Profile. Gemeinsamer Kern, deklarative Presets, Capability-Abhängigkeiten und ein profilabhängiges Tooling adressieren Wiederverwendung, Standardisierung und Flexibilität. Die dokumentierte Abnahme stützt Generierung, Installation, Tests und Builds im dort untersuchten Umfang; besonders stark ist diese Evidenz für die fünf Scaffold-Varianten, die kompatiblen PostgreSQL-Kombinationen und das Linux-Packaging (kleiveist, 2026j). Die späteren externen Läufe ergänzen aktuelle Web-, Container-, Linux- und macOS-Nachweise, enthalten aber zugleich reale Fehler in Tooling-, Desktopprofil- und Windows-Jobs (Abschnitt 6.2).

Dem stehen höhere zentrale Pflegekomplexität, manuell synchronisierte Schnittstellenartefakte und fehlende automatische Updates bereits generierter Projekte gegenüber. Plattform- und Betriebsnachweise enden vor einem grünen Windows-Paket, signierten Windows-/macOS-Releases, Notarisierung und produktivem Cloudbetrieb. Diese Punkte begrenzen die technische Verifikation, ohne die bewusst provider- und fachlogikfreie Baseline als fehlerhaft einzustufen.

Das Produktivitätspotenzial ist deshalb plausibel, aber nicht quantitativ belegt: Automatisierung und

gemeinsame Workflows können wiederkehrenden Setup-Aufwand reduzieren, während Template-Pflege und Technologievielfalt einen Teil dieses Nutzens aufzehren. Insgesamt eignet sich der Stand als standardisierte Ausgangsbasis für die vorgesehenen Projektfamilien, nicht als fertige Produktionsanwendung oder als universelle Vorlage. Die endgültige Beantwortung der Forschungsfragen und die Übertragung auf konkrete Einsatzfelder bleiben den nachfolgenden Kapiteln vorbehalten.

8 Einsatzmöglichkeiten und Transfer auf Kundenprojekte

8.1 Geeignete Projekttypen

Die Projekteignung wird aus dem Zielbild in Kapitel 2, den vorhandenen Architekturbausteinen, dem Verifikationsstand und den in Kapitel 7 zusammengeführten Stärken und Grenzen abgeleitet. MaSS-geblich sind die benötigten Zugriffskanäle, Backend- und Persistenzbedarf, Deploymentform, Umfang nativer Integration sowie Abweichungen bei Infrastruktur, Sicherheit und Betrieb. Direkt belegt sind die generierbaren Feature-Kombinationen und die im technischen Abnahmeprotokoll geprüften Profilpfade. Die Einstufung eines Projekttyps als *gut geeignet*, *bedingt geeignet* oder *nicht primär vorgesehen* ist demgegenüber eine qualitative Transferbewertung: Sie drückt den Abstand der Anforderungen zur Baseline aus, nicht eine empirisch ermittelte Erfolgswahrscheinlichkeit (kleiveist, 2026g, 2026j).

Gut geeignet erscheinen zunächst browserbasierte Frontends, die mit dem Profil `web-only` ohne Template-Backend ausgeliefert werden können. Benötigt eine Webanwendung zusätzlich eine zentrale API und eine providerneutrale Deploymentstruktur, deckt `web-cloud` diese Grenze mit Vite, TypeScript, FastAPI und den Cloud-Bausteinen ab. Hierzu zählen beispielsweise interne Business-Anwendungen, Verwaltungsoberflächen und datenbasierte Webanwendungen. Relationale Persistenz entsteht jedoch erst durch die separate Capability `postgres`; Authentifizierung, Berechtigungen, Fachmodelle und produktive Infrastruktur bleiben produktspezifisch.

Für lokale Desktop-Werkzeuge stellt `desktop-local` das gemeinsame Frontend in einer Tauri-Laufzeit bereit. Die Eignung gilt insbesondere, wenn kein zentrales Backend erforderlich ist und benötigte native Funktionen mit überschaubaren Tauri-Kommandos ergänzt werden können; eine lokale Datenbank gehört nicht zur Baseline. `desktop-cloud` ergänzt dagegen FastAPI und Deployment-Bausteine für einen Desktop-Client mit zentral bereitgestellten Diensten. Erfordert ein Produkt Browser- und Desktopzugang auf derselben Frontend- und Backendbasis, bildet `full-platform` den passenden semantischen Ausgangspunkt. Auch dort bleibt PostgreSQL optional. Der primäre Einsatzbereich lässt sich daher als Web-, Cloud- und Desktop-orientierte Business-/Full-Stack-Anwendungen zusammenfassen, sofern ihre technischen Anforderungen innerhalb dieser Grenzen liegen.

Kategorie	Projekttypen	MaSSgebliches Kriterium
GUT GEEIGNET	Browser-Frontend; Web + Backend/Cloud; lokales Desktop-Werkzeug; Desktop + Cloud; Web + Desktop + Cloud	Zugriffskanäle und Laufzeitschichten entsprechen einem geprüften Profil; Fach- und Betriebsfunktionen bleiben zu ergänzen.
BEDINGT GEEIGNET	hoher nativer Anteil; stark individualisierte Infrastruktur; hohe Compliance- oder Sicherheitsauflagen	Ein Profil deckt nur einen Teil ab; erhebliche Erweiterungen oder zusätzliche Nachweise sind erforderlich.
NICHT PRIMÄR VORGESEHEN	harte Echtzeitsysteme; Game-Engine-basierte Spiele; native Mobile-Apps; Firmware- und Embedded-Kernsysteme	Die tragende Laufzeit- oder Plattformarchitektur liegt außerhalb der Vite-/FastAPI-/Tauri-Baseline.

Abbildung 17: Qualitative Eignungsübersicht nach Architekturfit und Anpassungsbedarf

Quelle: Eigene Transferbewertung auf Grundlage der Profildefinitionen und der technischen Abnahme (kleiveist, 2026g, 2026j).

Abbildung 17 verdichtet die Einstufung; Tabelle 10 ordnet ihr Profile und wesentliche Ergänzungen zu. *Geeignet* bedeutet dabei nur, dass die Baseline einen relevanten Teil der Plattformarchitektur abdeckt. Ein fertiges Produkt entsteht daraus erst durch Fachlogik, Benutzeroberfläche, Integrationen, Produktsicherheit und den konkreten Betrieb. Technische Eignung ist zudem nicht mit Wirtschaftlichkeit gleichzusetzen; Budget, Teamkompetenzen, Betriebskosten und Produktstrategie sind nicht Gegenstand dieser Transferbewertung.

Tabelle 10: Zuordnung technischer Projekttypen zur Template-Baseline

Projekttyp	Eignung	Profil	Notwendige Ergänzungen und Grenzen
Browser-Frontend	gut	web-only	Fachlogik und UI; externe APIs möglich, aber kein FastAPI-Backend der Baseline.
Webanwendung mit API/Cloud	gut	web-cloud	Authentifizierung, Fachmodelle und produktiver Betrieb; PostgreSQL nur bei relationalem Persistenzbedarf.
lokales Desktop-Werkzeug	gut	desktop-local	Native Kommandos, Packaging und Signing produktspezifisch; keine lokale Datenbank enthalten.
Desktop mit zentralem Backend	gut	desktop-cloud	Produktive API, Security und Signing; PostgreSQL bleibt optional.
Browser + Desktop + Cloud	gut	full-platform	Kanalbezogene UX und Tests; Persistenz nur mit separater Capability.
Anwendung mit hohem nativen Anteil	bedingt	Desktopprofil oder andere Basis	Erheblicher Aufwand für Plattform-APIs, Berechtigungen, systemnahe Dienste oder native UI.
stark spezialisierte Infrastruktur	bedingt	nach Teilarchitektur	Eignung nur, wenn Client oder Einzeldienst sinnvoll abgrenzbar ist; Infrastruktur nicht vorgegeben.
Hard Real-Time, Game Engine, native Mobile- oder Embedded-Kernsysteme	nicht primär vorgesehen	kein Profil	Tragende Laufzeit- und Plattformanforderungen liegen außerhalb der Baseline.

Quelle: Eigene qualitative Zuordnung auf Grundlage von kleiveist (2026g) und kleiveist (2026j).

8.2 Ungeeignete und eingeschränkt geeignete Projekttypen

Eingeschränkt geeignet sind Vorhaben, deren Zugriffskanal zwar einem Profil entspricht, deren Kern aber deutlich andere technische Eigenschaften verlangt. Bei umfangreicher Betriebssystemintegration, plattformspezifischer UI, systemnahen Diensten oder Treibern reicht die vorhandene Tauri-Hülle nicht als Nachweis der Eignung; native Kommandos, Berechtigungen und Tests müssten in erheblichem Umfang produktspezifisch ergänzt werden. Gleiches gilt für eine zwingend abweichende Infrastruktur, etwa eine stark verteilte Microservice-Landschaft, spezielle Event-Streaming- oder HPC-Anforderungen. Hier ist zu prüfen, ob die Baseline als abgegrenzter Client oder Dienst weiterverwendet werden kann oder eine andere Architektur zweckmäßiger ist.

Nicht zum primären Zielbereich gehören harte Echtzeitsysteme, deren Korrektheit von garantierten Antwortzeiten abhängt, grafikintensive Spiele mit Bedarf an einer Game Engine sowie Firmware-, Mikrocontroller- und treiberzentrierte Embedded-Systeme. Gewöhnliche Live-Aktualisierungen über HTTP oder WebSockets sind davon zu unterscheiden: Sie machen eine Business-Anwendung noch nicht zu einem Hard-Real-Time-System. Native iOS- und Android-Anwendungen sind ebenfalls kein vorhandenes Standardprofil. Diese Abgrenzungen bedeuten nicht, dass jede spielähnliche, mobile angebundene oder hardwarenahe Teilfunktion technisch unmöglich wäre; ihr jeweiliger Kern wird durch die untersuchte Baseline jedoch nicht bereitgestellt.

Hohe Sicherheits- oder Compliance-Anforderungen führen nicht automatisch zu einer negativen Einstufung. Die Baseline allein weist aber weder produktspezifische Schutzbedarfe noch Auditierung, Zertifizierung oder Rechtskonformität nach. Der in Kapitel 6 dokumentierte Stand belegt zwar Compose-Validierung und Master-Image-Builds, nicht jedoch einen Compose-Start oder produktive Bereitstellung (GitHub, 2026a). Linux- und macOS-Pakete wurden unsigned gebaut; der Windows-Paketpfad ist fehlgeschlagen (GitHub, 2026c). Signing und Notarisierung bleiben produktspezifische Prüfgegenstände (kleiveist, 2026h). Solche Projekte erfordern daher zusätzliche Nachweise. Bei der Einsatzentscheidung ist ferner zu berücksichtigen, ob im Team die für die gewählte Variante benötigten TypeScript-, Python-, Rust-, Datenbank- und Deploymentkompetenzen vorhanden sind.

8.3 Analyse von Kundenanforderungen

Die Analyse beginnt technikneutral mit dem fachlichen Ziel, den Nutzergruppen, Arbeitsabläufen und der Einsatzumgebung. Erst danach werden funktionale Anforderungen in technische Entscheidungsdimensionen übersetzt. Die Aussage, mehrere Beschäftigte sollten denselben Datenbestand bearbeiten, beschreibt beispielsweise zunächst eine fachliche Zusammenarbeit. Daraus können ein zentrales Backend und dauerhafte Persistenz folgen; PostgreSQL ist damit aber noch nicht ohne Prüfung als Lösung festgelegt.

Anschließend werden Browser- und Desktopzugang, zentrale serverseitige Funktionen, relationale Persistenz und Offline-Bedarf getrennt erfasst. Hinzu kommen erforderliche native Funktionen, externe APIs, Betriebs- und Deploymentumgebung sowie Sicherheits- und Compliance-Vorgaben. Gerade der Offline-Fall ist gesondert zu behandeln: `desktop-local` benötigt zwar kein Cloud-Backend, enthält aber weder automatisch ein lokales Datenmodell noch eine Synchronisationslogik. Tabelle 11 fasst die Leitfragen und ihren Einfluss auf die Auswahl zusammen.

Tabelle 11: Kompakte Checkliste zur technikneutralen Anforderungsanalyse

Bereich	Leitfrage	Einfluss auf die Auswahl
Fachfunktion und Nutzung	Welche Abläufe, Nutzergruppen und Einsatzorte sind abzudecken?	Bestimmt Produktumfang; noch keine Profilstellung.
Zugriffskanäle	Wird das Produkt im Browser, als Desktop-Anwendung oder über beide Kanäle genutzt?	Trennt Web-, Desktop- und Full-platform-Pfade.
Zentrale Dienste	Müssen Funktionen oder gemeinsam genutzte Daten serverseitig bereitstehen?	Erfordert ein backendfähiges Cloudprofil.
Persistenz und Offline-Betrieb	Werden zentrale relationale Daten benötigt; muss der Client ohne Netz arbeitsfähig sein?	PostgreSQL nur bei Backendbedarf; lokale Speicherung und Synchronisation sind separate Ergänzungen.
Native Integration	Welche Betriebssystem-APIs, Geräte oder plattformspezifischen Oberflächen sind notwendig?	Bestimmt Tauri-Bedarf und Abstand zur nativen Baseline.
Integration und Betrieb	Welche Fremdsysteme, Zielumgebungen, Skalierungs- und Betriebsanforderungen bestehen?	Erzeugt produktspezifische Adapter und Infrastrukturentscheidungen.
Security und Compliance	Welche Schutz-, Nachweis-, Audit- oder regulatorischen Vorgaben gelten?	Erfordert zusätzliche MaSSnahmen und Evidenz unabhängig vom Profil.

Quelle: Eigene Darstellung.

Das Analyseergebnis enthält damit nicht nur eine Liste benötigter Plattformkomponenten, sondern auch einen Abweichungskatalog. Darin werden beispielsweise Authentifizierung und Rollen, konkrete Datenmodelle, Drittanbieterintegrationen, Observability, Backups, produktive Infrastruktur oder Desktop-Signing erfasst, soweit sie für das Produkt relevant sind. Erst auf dieser Grundlage lässt sich entscheiden, ob ein Profil passt, eine begrenzte Erweiterung notwendig ist oder die Anforderung außerhalb der Baseline liegt.

8.4 Auswahl des Projektprofils

Die Profilwahl übersetzt die analysierten Zugriffskanäle in das in Kapitel 4 definierte Modell. Für ein reines Browser-Frontend ohne Template-Backend wird `web-only` gewählt; Browser, FastAPI-Backend und Deploymentstruktur führen zu `web-cloud`. Ein lokaler Desktop-Client ohne Cloud-Backend entspricht `desktop-local`, ein Desktop-Client mit Backend und Cloud-Bausteinen `desktop-cloud`. Werden zusätzlich Browser- und Desktopkanal als Produktkanäle benötigt, ist `full-platform` vorgesehen. Da die aktuellen Presets Backend und Cloud gemeinsam aktivieren, existiert kein separates Profil für ein Template-Backend ohne die providerneutralen Deployment-Dateien. Der Entscheidungsfluss in Abbildung 8 stellt diese Zuordnung bereits dar.

Ausgewählt wird das kleinste Profil, das die tatsächlich benötigten Kanäle und Serverkomponenten abdeckt. `full-platform` ist daher keine allgemeine Vorzugsvariante: Nicht benötigte Komponenten würden zusätzliche Abhängigkeiten, Prüfpfade und Ausgangskomplexität in das Produkt übernehmen. Nach der Profilwahl wird Persistenz als getrennte Achse geprüft. Nur wenn eine backendfähige Variante relationale PostgreSQL-Persistenz benötigt, wird die Capability `postgres` ergänzt; sie löst die interne `database`-Abhängigkeit auf. Für `web-only` und `desktop-local` ist diese Kombination nicht zulässig (kleiveist, 2026g).

Die Entscheidung wird in einer von drei Formen dokumentiert: Das Profil deckt die Plattformanforderungen vollständig ab, es passt mit benannten produktspezifischen Erweiterungen oder ein zentraler Teil liegt außerhalb der Baseline. Im letzten Fall darf nicht ersatzweise das grösste Profil

gewählt werden; vielmehr ist eine alternative Architektur oder eine klar abgegrenzte Teilnutzung zu untersuchen. Wirtschaftliche Randbedingungen und verfügbare Teamkompetenzen ergänzen diese technische Entscheidung, verändern aber nicht die deklarierte Bedeutung der Profile.

8.5 Generierung eines Kundenprojekts

Aus der Auswahl ergeben sich die Eingaben für die in Kapitel 5 beschriebene Generierung: Profil, optionale Capability, Projektname, gegebenenfalls Slug, Zielverzeichnis und bei angepasster Desktop-Identität ein Reverse-Domain-Identifier. `init` löst daraus die Feature-Pfade auf und erzeugt ein separates Projekt mit aktivem `project-profile.toml`. Dieser Schritt ist der Übergabepunkt zwischen Template-Auswahl und Produktentwicklung, nicht die Fertigstellung eines Kundenprodukts (kleiveist, 2026g, 2026l).

Vor Beginn der Fachentwicklung wird die erzeugte Baseline im Zielverzeichnis geprüft. Der Standardweg umfasst `doctor`, `install`, `run` und `test --suite all`; profilabhängig kommen etwa `tauri doctor`, Datenbankdiagnose oder Container-Validierung hinzu. Damit wird kontrolliert, ob Manifest, Abhängigkeiten und aktivierte Schichten technisch konsistent sind. Der Ablauf wiederholt nicht die Implementierung des Generators, sondern ordnet dessen Ergebnis in den vollständigen Transferprozess der Abbildung 18 ein.

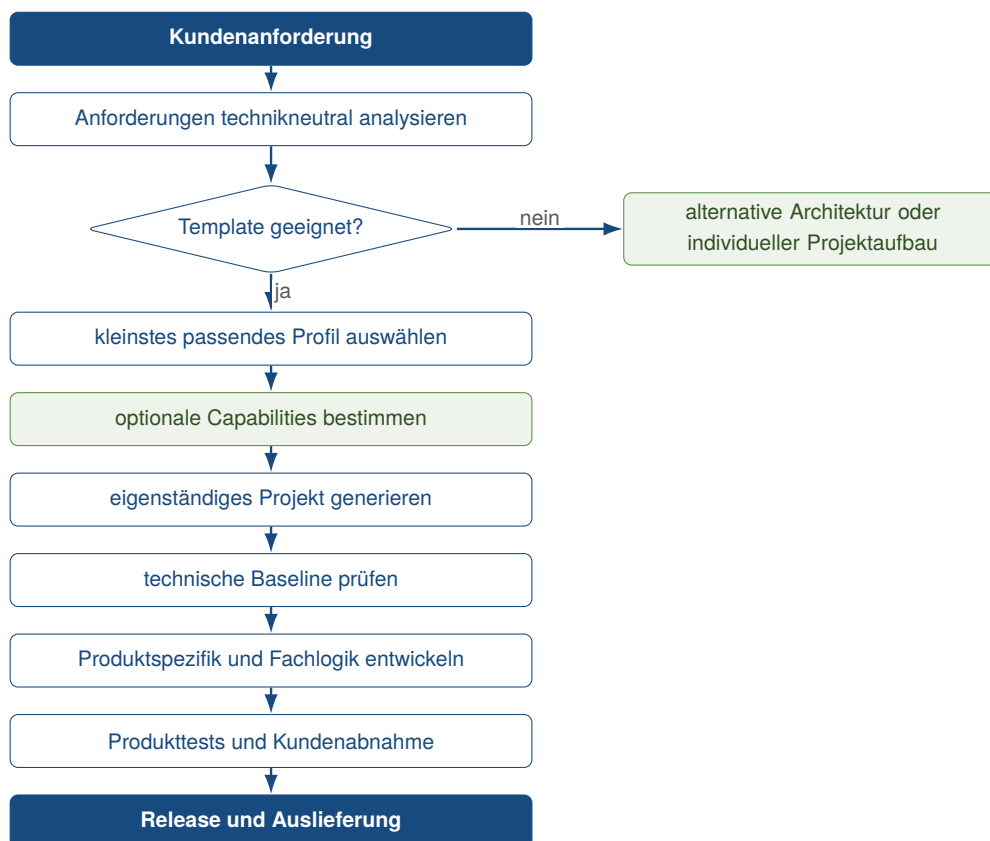


Abbildung 18: Vertikaler Transferprozess von der Kundenanforderung zum Produktprojekt

Quelle: Eigene Darstellung.

8.6 Überführung in die Produktentwicklung

Nach der Generierung bildet das Zielverzeichnis ein eigenständiges Produktprojekt in einem separaten Repository. Das Master-Template verantwortet die wiederverwendbare technische Baseline und deren Ableitung; das Produktprojekt übernimmt ab diesem Punkt Fachmodelle, Geschäftsregeln, Benutzeroberfläche, externe Integrationen, Produktdaten und Betriebsparameter. Hinzu kommen je nach Anforderung Architekturentscheidungen zu Authentifizierung und Berechtigungen, Skalierung, Monitoring, Backup, Datenschutz oder Desktop-Signing. Diese Aufgaben werden durch die vorhandene Projektstruktur unterstützt, aber nicht automatisch gelöst.

Master-Template und Produktprojekt besitzen anschließend getrennte Lebenszyklen. Der untersuchte Generator kopiert ausgewählte Pfade, bietet aber keinen dauerhaften Update- oder Synchronisationsmechanismus. Spätere Verbesserungen des Masters gelangen daher nicht automatisch in bereits erzeugte Projekte; sie müssen hinsichtlich Nutzen, Kompatibilität und Produktrisiko bewertet und gezielt übernommen werden. Umgekehrt gehören produktspezifische Änderungen nicht ohne Generalisierungs- und Verifikationsprüfung zurück in die gemeinsame Baseline.

Schließlich sind technische Template-Abnahme und Produktabnahme zu trennen. Die commitgebundene Abnahme belegt für die Baseline unter dokumentierten Bedingungen unter anderem Generierung, Installation, Tests und ausgewählte Buildpfade. Sie schließt produktive Bereitstellung, Signing sowie produktspezifische Fach- und Sicherheitsanforderungen ausdrücklich aus (Kleiveist, 2026j). Die Produktabnahme muss deshalb die konkreten Kundenanforderungen, Geschäftsregeln, Integrationen und Betriebsbedingungen mit eigenen Tests und Nachweisen prüfen. Kapitel 800 überführt die technische Fallstudie somit in ein systematisches Auswahl- und Generierungsverfahren, ohne daraus Kundenerfolge oder eine universelle Eignungszusage abzuleiten.

9 Schlussbetrachtung

9.1 Zusammenfassung der Ergebnisse

Ausgangspunkt der Fallstudie war der wiederkehrende Aufwand für technisch ähnliche Projektinitialisierungen und das Risiko voneinander abweichender Baselines für Web-, Cloud- und Desktopanwendungen. Die daraus abgeleiteten Anforderungen bündeln Generierbarkeit und nachvollziehbare Konfiguration, konsistente Verantwortungsgrenzen, kontrollierte Varianten, gemeinsame Arbeitsabläufe und zentrale Wartbarkeit. Die Lösung setzt sie durch ein Master-Template mit gemeinsamem Kern und deklarativer Variabilität um.

Architektonisch verbindet die Baseline ein gemeinsames Vite-Frontend mit einem profilabhängigen FastAPI-Backend und einer optionalen Tauri-Schicht. PostgreSQL lässt sich nur bei Backendprofilen als Capability ergänzen. Fünf Projektprofile wählen daraus kontrollierte Presets; gemeinsame Contracts, Konfiguration und Tooling verbleiben im Kern. `control.py` bündelt Generierung, Diagnose, Installation, Entwicklungsbetrieb, Tests, Builds und Release-Vorbereitung. Generierte Produktprojekte bleiben eigenständig und erhalten spätere Änderungen des Master-Templates nicht automatisch.

Die technische Evidenz ist nach Stand und Umgebung zu unterscheiden. Die dokumentierte lokale Abnahme eines früheren Commits weist den Status *PASS WITH OPEN EXTERNAL ITEMS* und *READY FOR CASE STUDY* aus und stützt die untersuchten Profil-, PostgreSQL-, Test-, Web-Build-

und Linux-Paketierungspfade. Die aktuelle externe CI ist dagegen nur teilweise erfolgreich; weitere Pfade sind nicht anwendbar, nicht verifiziert oder produktspezifisch offen. Das Ergebnis ist eine standardisierte, kontrolliert variable Baseline mit plausiblen Produktivitätspotenzial, aber auch zentraler Pflegekomplexität, manueller Schnittstellensynchronisierung und unvollständiger Plattform- und Betriebsevidenz. Produktionsreife oder empirisch bestätigte Kundeneignung lassen sich daraus nicht ableiten.

9.2 Beantwortung der Fragestellungen

Fragestellung 1: Adressierte Anforderungen. Das Technologie-Template adressiert die in Kapitel 2 formulierten fachlich-technischen, qualitativen und betrieblichen Anforderungen durch eine generierbare Baseline, klare Komponentenstrukturen, profilabhängige Auswahl, gemeinsame Workflows, zentrale Pflege sowie versionierte Konfiguration und Capability-Abhängigkeiten. Getrennte Laufzeitgrenzen und profilabhängige Prüfpfade stützen dies. Fachlogik, Authentifizierung, Berechtigungen und Produktivbetrieb bleiben produktspezifisch.

Fragestellung 2: Varianten für Web, Cloud und Desktop. Architektur und Profilmodell kombinieren den gemeinsamen Frontend-Kern mit optionalen Backend-, Cloud- und Tauri-Bausteinen. Fünf Presets bilden die Plattformzuschnitte ab; PostgreSQL wird getrennt und nur bei Backendfähigkeit aufgelöst. Generierte Projekte enthalten so überwiegend nur benötigte Pfade. Dies gilt für den untersuchten Katalog; `desktop-cloud` und `full-platform` sind technisch gleich gescaffoldet und vor allem semantisch abgegrenzt.

Fragestellung 3: Wiederverwendung. Gemeinsamer Kern und Single-Repository-Strategie fördern Wiederverwendung und Standardisierung weitgehend, weil Implementierungen, Profile, Generator, Konfiguration, Tests und Dokumentation zentral gepflegt werden. Deklarative Presets und gemeinsame Tooling-Befehle begrenzen abweichende Ausgangsstände. Dieser Vorteil gilt unmittelbar für die Baseline und künftige Generierungen; bereits abgeleitete Produktprojekte bleiben separate Repositories ohne automatischen Rückfluss späterer Template-Änderungen.

Fragestellung 4: Reduktion wiederkehrender Aufwände. Für Projektinitialisierung, Entwicklungsworkflow, Qualitätssicherung und Bereitstellung besteht begründbares Einsparpotenzial: Profilwahl, Generierung, Diagnose, Installation, Start, Tests und Builds sind automatisiert oder vereinheitlicht und teilweise technisch abgenommen. Messdaten zu Zeit und Aufwand sowie Vergleichsdaten fehlen. Eine Produktivitätssteigerung ist daher nicht nachgewiesen; zudem sind zentrale Pflege und mehrere Toolchains gegenzurechnen.

Fragestellung 5: Projekteignung. Eine geeignete Ausgangsbasis stellt das Template grundsätzlich für technisch verwandte Web-, Desktop- und kombinierte Business-Anwendungen dar, deren Plattform-, Cloud- und Persistenzbedarf abbildbar ist. Stark native Anwendungen, Spiele und spezialisierte Echtzeitsysteme liegen nicht im primären Zielbereich. Kapitel 8 leitet eine technikneutrale Analyse, das kleinste passende Profil und die getrennte Capability-Auswahl ab. Die Einstufung bleibt qualitativ und ist keine universelle Eignungszusage.

Fragestellung 6: Grenzen, Wartungsaufwände und Risiken. Zu berücksichtigen sind die wachsende Regressionsbreite des Master-Templates, manuell synchronisierte Schnittstellen, die nicht strikt getrennte Capability-Auswahl und fehlende automatische Updates generierter Projekte. Mehrere Toolchains erhöhen den Pflegebedarf. Aktuell schlagen Tooling-, Desktopprofil- und Windows-CI-Pfade fehl; Release Validation, Compose-Start und Branch Protection sind nicht verifiziert. Signing, Notarisierung, Updater, Produktiv-Deployment und Sicherheitsmodell bleiben Produktaufgaben.

9.3 Kritische Würdigung

Die technische Fallstudie untersucht das konkrete Template detailliert, ist aber nur begrenzt generalisierbar. Projektunterlagen belegen die Implementierung, nicht deren unabhängige Evaluation; Abnahme und lokale Prüfung stammen ebenfalls aus dem Projektkontext. Die Verbindung von Entwicklerperspektive, Selbstdokumentation und Bewertung kann die Evidenzauswahl beeinflussen, ohne commitgebundene Befunde aufzuheben. Wissenschaftliche Literatur ordnet allgemeine Konzepte ein, offizielle Dokumentation die Technologien; eine unabhängige Replikation ersetzen beide nicht.

Die Testevidenz ist heterogen: Einer früheren, überwiegend Linux-basierten Abnahme stehen gemischte aktuelle Remote-Ergebnisse gegenüber. Jeder Test und Build belegt nur sein Kriterium; ohne Coverage-Messung, Compose-Start und durchgehend erfolgreiche Plattformläufe entsteht kein umfassender Qualitäts- oder Produktionsnachweis. Unsignierte Pakete ersetzen zudem keine nativen Produktreleases einschließlich Signierung und Notarisierung.

Übertragbar erscheinen die Prinzipien zentraler Baseline, deklarativer Variabilität und gemeinsamer Tooling-Einstiege; projektspezifisch bleiben Technologien, Profile und commitgebundene Evidenz. Produktivitätspotenzial und Projekteignung wurden qualitativ hergeleitet. Zeitmessungen, Vergleichsgruppen, reale Kundenfallprüfungen und eine wirtschaftliche Bewertung fehlen.

9.4 Ausblick

Kurzfristig sind zunächst die fehlgeschlagenen aktuellen Tooling-, Desktopprofil- und Windows-CI-Pfade zu korrigieren und auf einem festen Commit erfolgreich zu wiederholen. Ergänzend stehen ein vollständiger Compose-Start mit Health- und Readiness-Nachweis, ein Lauf der Release Validation sowie die autorisierte Prüfung der Branch Protection aus. Diese Schritte würden die offene technische Evidenz erweitern, ohne bereits Produktreife zu belegen.

Mittelfristig sind für ein konkretes Produkt Signierung, Notarisierung, Updater, Credentials, Deployment und Authentifizierung beziehungsweise RBAC separat auszugestalten und abzunehmen. Als mögliche Weiterentwicklung der Baseline lassen sich aus den festgestellten Grenzen eine stärkere Automatisierung gemeinsamer Contracts, eine striktere Capability-Auswahl und ein kontrolliertes Synchronisationsmodell für bereits generierte Projekte ableiten.

Langfristig sollte das Produktivitätspotenzial vergleichend über mehrere Projekte untersucht werden, etwa anhand von Onboarding-Zeit, Zeit bis zum ersten lauffähigen Build, manuellen Setup-Schritten, Fehlerhäufigkeit und Pflegeaufwand. Reale Kundenprojekte könnten zugleich prüfen, ob die analytisch abgeleiteten Profil- und Eignungskriterien tragen und wo zusätzlicher Anpassungsbedarf entsteht.

9.5 Fazit

Die in Abschnitt 1.3 formulierte Zielsetzung wurde hinsichtlich der strukturierten technischen Untersuchung und qualitativen Bewertung weitgehend erreicht. Die Fallstudie zeigt, dass ein gemeinsamer Kern, deklarative Profile, getrennte Capabilities und ein zentraler Tooling-Einstieg eine konsistente, wiederverwendbare Baseline für die definierten Web-, Cloud- und Desktopvarianten bilden können.

Der wesentliche Nutzen liegt in kontrollierter Variabilität und der Vereinheitlichung wiederkehrender Projekt- und Entwicklungsabläufe. Daraus ergibt sich ein plausibles Produktivitätspotenzial, dessen tatsächliche Grösse ohne empirische Zeit- und Aufwandsdaten offen bleibt. Ebenso ist die Übertragung auf Kundenprojekte bislang nur für den technisch abgegrenzten Zielbereich analytisch begründbar.

Das Gesamturteil endet daher bei der Reife einer untersuchten technischen Template-Baseline. Die lokale Abnahme stützt den dafür betrachteten Stand; aktuelle CI-Fehler, offene externe Prüfungen und produktspezifische Auslieferungs- und Sicherheitsaufgaben verhindern jedoch eine Gleichsetzung mit einem fertigen, plattformübergreifend verifizierten Produkt. Gerade diese Trennung bestimmt die Aussagekraft der Fallstudie.

Literaturverzeichnis

- Bayer, M. (2026). *Alembic tutorial* [Alembic 1.19.1 Documentation]. Verfügbar 8. August 2026 unter <https://alembic.sqlalchemy.org/en/latest/tutorial.html>
- Clements, P. C., & Northrop, L. M. (2001). *Software product lines: Practices and patterns*. Addison-Wesley Professional. Verfügbar 8. August 2026 unter <https://www.sei.cmu.edu/library/software-product-lines-practices-and-patterns/>
- Forsgren, N., Storey, M.-A., Maddila, C., Zimmermann, T., Houck, B., & Butler, J. (2021). The space of developer productivity: There's more to it than you think. *Queue*, 19(1), 20–48. <https://doi.org/10.1145/3454122.3454124>
- GitHub. (2026a, 7. August). *Core ci, run 31168215013* [GitHub-Actions-Lauf für Commit a4487ac im Repository Template-Projekte]. Verfügbar 9. August 2026 unter <https://github.com/kleiveist/Template-Projekte/actions/runs/31168215013>
- GitHub. (2026b). *Creating a repository from a template* [GitHub Documentation]. Verfügbar 9. August 2026 unter <https://docs.github.com/en/repositories/creating-and-managing-repositories/creating-a-repository-from-a-template>
- GitHub. (2026c, 7. August). *Desktop ci, run 31168214763* [GitHub-Actions-Lauf für Commit a4487ac im Repository Template-Projekte]. Verfügbar 9. August 2026 unter <https://github.com/kleiveist/Template-Projekte/actions/runs/31168214763>
- GitHub. (2026d, 7. August). *Postgresql integration, run 31168216208* [GitHub-Actions-Lauf für Commit a4487ac im Repository Template-Projekte]. Verfügbar 9. August 2026 unter <https://github.com/kleiveist/Template-Projekte/actions/runs/31168216208>
- GitHub. (2026e, 7. August). *Profile matrix, run 31168215737* [GitHub-Actions-Lauf für Commit a4487ac im Repository Template-Projekte]. Verfügbar 9. August 2026 unter <https://github.com/kleiveist/Template-Projekte/actions/runs/31168215737>
- ISO/IEC. (2023). *Systems and software engineering—systems and software quality requirements and evaluation (square)—product quality model* (International Standard ISO/IEC 25010:2023). International Organization for Standardization. Verfügbar 8. August 2026 unter <https://www.iso.org/standard/78176.html>
- Klabnik, S., Nichols, C., & Krycho, C. (2026). *The rust programming language* [Official Rust Documentation]. Verfügbar 8. August 2026 unter <https://doc.rust-lang.org/book/>
- kleiveist. (2026a, 7. August). *Central project control cli* [CLI-Implementierung im Repository Template-Projekte]. Verfügbar 9. August 2026 unter <https://github.com/kleiveist/Template-Projekte/blob/a4487acfd6db896202009ff6c13567c319dfdb/tools/control.py>
- kleiveist. (2026b, 7. August). *Container builds and local production simulation* [Projektdokumentation im Repository Template-Projekte]. Verfügbar 9. August 2026 unter <https://github.com/kleiveist/Template-Projekte/blob/a4487acfd6db896202009ff6c13567c319dfdb/docs/tools/container-builds.md>
- kleiveist. (2026c, 7. August). *Continuous integration* [Projektdokumentation im Repository Template-Projekte]. Verfügbar 9. August 2026 unter <https://github.com/kleiveist/Template-Projekte/blob/a4487acfd6db896202009ff6c13567c319dfdb/docs/tools/ci.md>
- kleiveist. (2026d, 6. August). *Database feature* [Projektdokumentation im Repository Template-Projekte]. Verfügbar 8. August 2026 unter <https://github.com/kleiveist/Template-Projekte/blob/a4487acfd6db896202009ff6c13567c319dfdb/docs/def/database-feature.md>

- kleiveist. (2026e, 6. August). *Deployment architecture* [Projektdokumentation im Repository Template-Projekte]. Verfügbar 8. August 2026 unter <https://github.com/kleiveist/Template-Projekte/blob/a4487acfcdf6db896202009ff6c13567c319dfdb/docs/def/deployment-architecture.md>
- kleiveist. (2026f, 6. August). *Framework architecture* [Projektdokumentation im Repository Template-Projekte]. Verfügbar 8. August 2026 unter <https://github.com/kleiveist/Template-Projekte/blob/a4487acfcdf6db896202009ff6c13567c319dfdb/docs/def/architecture.md>
- kleiveist. (2026g, 6. August). *Project profiles* [Projektdokumentation im Repository Template-Projekte]. Verfügbar 8. August 2026 unter <https://github.com/kleiveist/Template-Projekte/blob/a4487acfcdf6db896202009ff6c13567c319dfdb/docs/def/project-profiles.md>
- kleiveist. (2026h, 7. August). *Release and desktop packaging model* [Projektdokumentation im Repository Template-Projekte]. Verfügbar 9. August 2026 unter <https://github.com/kleiveist/Template-Projekte/blob/a4487acfcdf6db896202009ff6c13567c319dfdb/docs/tools/release-model.md>
- kleiveist. (2026i, 6. August). *Runtime configuration* [Projektdokumentation im Repository Template-Projekte]. Verfügbar 8. August 2026 unter <https://github.com/kleiveist/Template-Projekte/blob/a4487acfcdf6db896202009ff6c13567c319dfdb/docs/def/configuration.md>
- kleiveist. (2026j, 7. August). *Template final acceptance* [Technisches Abnahmeprotokoll im Repository Template-Projekte]. Verfügbar 8. August 2026 unter <https://github.com/kleiveist/Template-Projekte/blob/a4487acfcdf6db896202009ff6c13567c319dfdb/docs/dev/template-final-acceptance.md>
- kleiveist. (2026k, 7. August). *Template-projekte: Full-stack project template* [GitHub-Repository, geprüfter Commit a4487ac]. Verfügbar 8. August 2026 unter <https://github.com/kleiveist/Template-Projekte>
- kleiveist. (2026l, 6. August). *Tooling guide* [Projektdokumentation im Repository Template-Projekte]. Verfügbar 8. August 2026 unter <https://github.com/kleiveist/Template-Projekte/blob/a4487acfcdf6db896202009ff6c13567c319dfdb/docs/tools/tooling.md>
- Krueger, C. W. (1992). Software reuse. *ACM Computing Surveys*, 24(2), 131–183. <https://doi.org/10.1145/130844.130856>
- Lamb, C., & Zacchiroli, S. (2022). Reproducible builds: Increasing the integrity of software supply chains. *IEEE Software*, 39(2), 62–70. <https://doi.org/10.1109/MS.2021.3073045>
- Microsoft. (2026, 27. Juli). *The typescript handbook* [TypeScript Documentation]. Verfügbar 8. August 2026 unter <https://www.typescriptlang.org/docs/handbook/intro.html>
- PostgreSQL Global Development Group. (2026). *Postgresql 18.4 documentation*. Verfügbar 8. August 2026 unter <https://www.postgresql.org/docs/current/index.html>
- Ramírez, S. (2026). *Fastapi features* [Official FastAPI Documentation]. Verfügbar 8. August 2026 unter <https://fastapi.tiangolo.com/features/>
- Runeson, P., & Höst, M. (2009). Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, 14(2), 131–164. <https://doi.org/10.1007/s10664-008-9102-8>
- SQLAlchemy Authors and Contributors. (2026, 15. Juni). *Engine configuration* [SQLAlchemy 2.0 Documentation, Release 2.0.51]. Verfügbar 8. August 2026 unter <https://docs.sqlalchemy.org/en/20/core/engines.html>
- Tauri Contributors. (2026a). *Capabilities* [Tauri 2 Documentation]. Verfügbar 8. August 2026 unter <https://v2.tauri.app/security/capabilities/>

Tauri Contributors. (2026b, 22. Juli). *What is tauri?* [Tauri 2 Documentation]. Verfügbar 8. August 2026 unter <https://v2.tauri.app/start/>

VoidZero Inc. and Vite Contributors. (2026). *Getting started* [Vite Documentation]. Verfügbar 8. August 2026 unter <https://vite.dev/guide/>